UNIVERSITY OF GREENWICH
FACULTY OF ENGINEERING AND SCIENCE
SCHOOL OF COMPUTING AND MATHEMATICAL SCIENCES

A Localised Framework for Assessing Windows Telemetry: Quantifying Data Exposure through Registry and Diagnostic Analysis

https://github.com/ipask-1225/Windows-Telemetry-Assessment-Tool.git

Ieva Paskeviciute (001303411)

Supervisor: Ehsan Nowroozi

Word count: 11,165

COMP1682 Final Year Individual Project

A dissertation submitted in partial fulfilment of the requirements for the degree of

BSc (Hons) Cyber Security and Digital Forensics

April 2026

ABSTRACT

This project addresses the privacy concerns regarding data telemetry specifically within the Windows operating system, with a locally developed application that analyses native Windows services, the 'Windows Telemetry Assessment Tool'. The developed application was the result of critical analysis of the Microsoft documentation on data collection (Microsoft, 2024) and published digital forensic investigative methodologies from Carvey (2011) and Casey (2011), to address the significant need for a tool with high-level forensic artefact examination. The tool analyses the filtered Windows registries responsible for processing user data, the Windows data diagnostic files, and filtered background services within the Windows Services Manager. Furthermore, the tool provides configuration details on how data on the local system is set to be collected, and the actual data collected through diagnostic files. The forensic examination approach provides a comprehensive view of the user's data exposure when using a Windows device.

DECLARATION OF AI USE

Please complete this part when you have used AI during the process of undertaking this assignment to acknowledge the ways in which you have used it.

I have used AI while undertaking my assignment in the following ways:

- To develop research questions on the topic – NO
- To create an outline of the topic – YES
- To explain concepts – NO
- To support my use of language – YES
- To summarise the following articles/resources: – NO
- In other ways, as described below: – YES

To provide structure to writing the sub-sections within the report, and paragraph layout within the main body text. (Chapters 1 – 6)

ACKNOWLEDGEMENTS

The completion of this project would have not been made possible without the emotional and academic support from my university peers across from courses in the Computing and Mathematical department. Their sustained support throughout the final year of my bachelor's course has been unfathomable, their encouragements and assistance allowed me to persevere, and stay motivated during challenging times. I thank them for their continued patience, resilience, and assistance in pushing me to achieve greater in my studies.

CONTENTS

Abstract ii
Declaration of AI Use iii
Acknowledgements iv
Contents v
1 Introduction 1
1.1 Background 2
1.2 Aims and Objectives 3
1.2.1 Objectives: 3
1.3 Methodology 4
1.4 Report Roadmap 5
2 Literature Review 6
2.1 Introduction 6
2.2 Telemetry within Modern Operating Systems 6
2.3 Privacy Concerns and Ethical Implications 7
2.4 Existing Privacy Tools 8
2.5 Conclusion 9
3 Requirements Analysis and Design 10
3.1 Introduction 10
3.2 Requirements Analysis 10
3.2.1 Design Requirements 10
3.2.2 Assumptions and Constraints 11
3.2.3 Functional Requirements 11
3.2.4 Non-functional Requirements 13
3.3 Design 14
3.3.1 System Architecture 14
3.3.2 Component Overview 16
3.3.3 Component Design 16
3.3.4 Registry Scanner Component 17
3.3.5 Services Scanner Component 18
3.3.6 Diagnostic Data Analyser Component 20
3.4 Conclusions 20
4 Implementation 21
4.1 Introduction 21

4.2 Development Environment and Technical Setup 21
4.3 Component Implementation 21
4.3.1 Registry Scanner Implementation 22
4.3.2 Technical Challenges 22
4.3.3 Services Scanner Implementation 23
4.3.4 Technical Challenge 24
4.3.5 Diagnostic Data Analyser Implementation 25
4.3.6 Exposure Score Calculator Implementation 26
4.4 Testing 28
4.4.1 Unit Testing 29
4.4.2 Integration Testing 29
4.4.3 Forensic Accuracy Validation 29
4.5 Conclusion 30
5 Results and Evaluation 31
5.1 Introduction 31
5.2 Results 31
5.2.1 Pre HushWin Configuration 31
5.2.2 Post-HushWin Configuration 32
5.2.3 Tool Accuracy Validation 33
5.3 Evaluation 34
5.3.1 Evaluation of Multi-Source Forensic Evidence Collection (Objective 1) 34
5.3.2 Evaluation of Quantified Risk Metrics (Objective 2) 35
5.3.3 Evaluation of User-Accessible Presentation (Objective 3) 35
5.3.4 Evaluation of Assessment-First Methodology 35
5.3.5 Limitation of the Accuracy Validation 36
5.4 Conclusion 36
6 Legal, Social, Ethical and Professional Issues 37
7 Conclusion 39
7.1 Summary of Windows Telemetry Assessment Tool Development 39
7.2 Summary of Overall Outcomes and Results 39
7.3 Reflection on Limitations and Challenges 40
7.4 Future Work 41
Reference List 42
APPENDIX A - Project Proposal 45
a.1 Revised Proposal 45
a.2 Original Proposal 45
Declaration of AI Use 47

Synopsis 48
1. Aim and Objectives 48
7.5 Aim 48
7.6 Objectives 49
7.6.1 Objective 1 49
7.6.2 Objective 2 49
7.6.3 Objective 3 49
7.6.4 Objective 4 49
7.6.5 Objective *5* 49
2. Background Research and Project Rationale 50
3. Methodology and Technical Approach 51
4. Evaluation Plan 52
5. LSEPi Considerations 53
6. Risk Assessment 54
7. Project Plan and Timeline 54
APPENDIX B - Implementation Details 57
b.1 Implemented Windows Services and Justification 57
b.2 Implemented Diagnostic Directories 61
b.3 Monolithic and Microservices Software Architecture Definitions 62

1 INTRODUCTION

Telemetry, as defined by Cambridge dictionary (2024), is "the science or process of collecting information about objects that are far away and sending the information somewhere electronically". Unlike specialised telemetry in controlled environments, Windows telemetry operates on general-purpose consumer devices that contains vast amounts of personal data, elevating privacy concerns beyond operational monitoring, to questions of data ownership and surveillance capitalism. This process has become a crucial method for data collection in modern technology, as a means to monitor events or metrics and to analyse system health for various purposes. In the context of information technology, telemetry is the information collected from the user or group of users with or without their consent by the developer or company (Tan et al., 2021). This is especially useful for software developers where they use information gathered from continuous streams of real-time data for a comprehensive view of performance. Whilst telemetry provides critical operational benefits to businesses, however, on the user level, privacy concerns have been raised over data security and the implications of the practises to collecting data. Data about the user platform such as the device type, location, and network are only examples of the collected information by an application service or business as a means for enhancing troubleshooting practices, or maintaining performance of a system. As noted by the Dutch Data Protection Authority (2017), the lack of transparency in telemetry levels and the purpose for the collected data can lead to a breach of trust, especially when telemetry has evolved from its original troubleshooting purpose to 'marketing' (Nissenbaum, 2009). Major corporations such as Microsoft increasingly rely on user information and insights for the same purpose of maintaining performance and troubleshooting, and as such the boundaries between necessary data collection and intrusive data gathering for commercial purposes have become a controversial topic of debate of data privacy.

Microsoft's operating system Windows 10 and 11 have their own method of collecting continuous streams of data, which they call Diagnostic Data (and will be referred as so throughout the project) primarily through a service named 'Connected User Experience and Telemetry' that operates at two levels: Required diagnostic data, and optional diagnostic data (Microsoft, 2024). Required diagnostic data is a mandatory service for home users that cannot be disabled through standard settings, and gathers information for 'product improvement' such as basic error information, data about performance of updates, and driver information. Overall, this service is what maintains a standard and functioning Windows device. Optional diagnostic data includes functions that analyse the users' search browser activity, device activity, product and service usage and more (Microsoft, 2024).

Despite the clear outline for why Microsoft implements their telemetry practices in improving product quality, the depth of the information collected within their optional diagnostic data has contributed to

numerous discussions regarding user privacy, and the depth to the amount of personally identifiable information (PII) collected and processed by Microsoft. Privacy advocates express their concerns over the default Windows settings, stating how it may inadvertently expose personal information or be used for purposes outside technical improvement, such as for commercial activity purposes, like targeted advertising (Bott, 2026). As such, it is crucial to understand what data is being processed, how it is being used and the services behind it in order to create a well-informed decision to protect user privacy in an increasingly data driven world.

1.1 Background

Windows 10 and Windows 11, Microsoft's flagship operating systems serving over 1.4 billion active devices globally (Statista, 2025), implement diagnostic data collection as a core system feature operating at an unprecedented scale in consumer software history. This telemetry infrastructure evolved from the Windows Error Reporting (WER) service introduced in Windows XP, which transmitted crash dumps reactively upon application failures, to the Connected User Experiences and Telemetry service (DiagTrack) in Windows 10, which proactively collects continuous streams of diagnostic data regardless of error states (Glerum et al., 2009; Radar, 2022). This architectural shift from reactive error reporting to proactive continuous monitoring represents a fundamental transformation in the relationship between operating system and the user, where users once controlled when data left their systems (by choosing whether to submit error reports), to modern telemetry infrastructure collecting data automatically, continuously, and without explicit per-event consent. As mentioned in Chapter 1, the introduction, Microsoft justifies this mandatory collection as "necessary to keep Windows secure, up to date, and functioning as expected" (Microsoft, 2024). This framing transforms privacy-conscious users from customers exercising their data choices, into the core system components whose data collection enables the product's basic functionality. While the choice of data collection within their Optional diagnostic data is marketed as an opt-in, Windows installation defaults to the Optional setting being enabled, requiring users to actively discover and modify their privacy settings; a pattern privacy researchers characterise as "dark patterns" designed to maximize data collection, while maintaining plausible deniability of informed consent (Maier and Harr, 2020).

The forensic examination of this telemetry infrastructure reveals three distinct artifact layers that persist within the local Windows installation. Configuration artifacts reside in Windows Registry hives, specifically ‘HKLM\SOFTWARE\Policies\Microsoft\Windows\DataCollection’ controlling telemetry levels and ‘HKLM\SOFTWARE\Microsoft\SQMClient\Windows’ governing the Customer Experience Improvement Program (Carvery, 2011). Behavioural artifacts manifest in running system services such as DiagTrack (Connected User Experiences and Telemetry), DPS (Diagnostic Policy Service) and

WerSvc (Windows Error Reporting). Content artifacts accumulate in diagnostic log files distributed across the following folder paths: C:\ProgramData\Microsoft\Diagnosis (centralized event traces), C:\Windows\System32\LogFiles (system operational logs), and user-specific directories like C:\ProgramData\Microsoft\Windows\WER (error reports with memory dumps). These artifacts create a local evidence trail of telemetry activity that standard users cannot inspect without specialised forensic tools. Carvey (2011) establishes that registry keys controlling telemetry settings often persist unchanged even when users modify Privacy Settings through the graphical interface, as Group Policy overrides and service-level configurations can supersede user-facing controls, a phenomenon Detmering and Ray (2025) term "configuration drift."

Current privacy tools fail to address this evidence gap. Microsoft's Diagnostic Data Viewer (introduced in Windows 10) provides transparency by displaying collected telemetry in JSON format, but this raw data presentation proves incomprehensible to non-technical, where data visibility substitutes for meaningful control (Charnsethikul et al., 2025). Conversely, third-party modification tools like O&O ShutUp10 (2024) and W10Privacy (BerndV, 2023) provide accessible toggle switches to disable telemetry services and registry keys, but operate on a modification paradigm that alters system state without first assessing current exposure level, or quantifying the privacy impact of existing configurations. Neither approach applies forensic principles of systematic evidence collection to quantify privacy exposure before intervention, leaving users unable to audit their own privacy assessment.

1.2 Aims and Objectives

This project aims to develop a forensic-based privacy assessment framework that quantifies Windows telemetry exposure through systematic examination of diagnostic artifacts, translating digital forensics methodology into a user-accessible tool that addresses the identified gap between expert forensic examination capabilities and end-user privacy assessment needs.

1.2.1 Objectives:

1. Multi-Source Forensic Evidence Collection: Design and implement three independent scanner components targeting distinct evidence types: registry configuration (AllowTelemetry, CEIP settings), service behaviour (12 telemetry-related services including DiagTrack, DPS, WerSvc), and diagnostic file content (PII detection across 10 storage locations including C:\ProgramData\Microsoft\Diagnosis and %LocalAppData%\Microsoft\Windows\WER).

2. Quantified Risk Metrics: Develop a weighted scoring algorithm producing 0-100 privacy exposure scores with transparent component breakdown (Registry: 30%, Services: 30%, Diagnostic Data: 40%),

addressing the critical gap in existing tools which provide either binary safe/unsafe classifications or uninterpreted raw data.

3. User-Accessible Presentation: Create a graphical user interface (GUI) using Windows Forms presenting forensic findings in non-technical language with progressive disclosure architecture enabling informed decision-making without requiring technical expertise.

4. Assessment-First Methodology: Implement read-only forensic examination strictly adhering to Casey's (2011) principle of documentation before intervention, fundamentally differentiating this tool from modification-first approaches (O&O ShutUp10, W10Privacy) that alter system state without prior assessment.

5. Forensic Validation and User Testing: Conduct forensic accuracy validation comparing tool results with manual expert examination across 10 test scenarios, and user acceptance testing with both technical and non-technical participants to verify comprehensibility and utility of quantified exposure metrics.

1.3 Methodology

This project adopted an iterative and prototyping development methodology, driven by the continuous repetitions of designing, implementation and testing in each cycle (Admin_CM_1, 2024). This method was selected for its suitability to a forensic tool where requirements evolve through testing and discovery rather than being fully defined upfront. The iterative approach allowed each scanner component to be built, tested and refined independently before integration, directly supporting the modular three-layered architecture defined in the design chapter (Chapter 3).

The research aspect of this project followed a qualitative secondary research methodology, drawing on existing academic literature, official technical documentation, and forensic investigation frameworks to establish the theoretical foundation for the tool's design. This was the most appropriate approach as the problem domain - telemetry within Windows architecture and digital forensic – is already a well-documented problem domain, and the project's contribution lies in applying existing forensic methodology to a new user-accessible context, rather than generating new empirical data about telemetry behaviour itself.

The development methodology was structured around Casey's (2011) forensic principle of documentation before intervention, which mandated the tool's assessment-first design throughout all iterations. This principle directly influenced every technical decision, ensuring that no component was permitted to write or modify system artefacts at any stage of development or testing. An agile-influenced

methodology was chosen over a traditional Waterfall model. Within the Waterfall model, testing and validation occur only after the full build is complete, presenting a critical risk for a forensic tool: logic errors or accidental write operations would not be discovered until final stages, invalidating the project's core objectives (Khan and Mahadik, 2022). Agile's continuous validation allowed each component to be verified immediately upon completion, ensuring the read-only constraint was maintained throughout. Visual Studio's MSTest framework was used for individual component validation alongside manual forensic validation on self-owned Windows devices. Git version control with frequent commits was maintained throughout to demonstrate incremental progress and provide rollback capability. The absence of formal user acceptance testing with external participants represents a known limitation, addressed in the evaluation chapter.

1.4 Report Roadmap

Chapter 2 reviews the academic literature on Windows telemetry architecture and existing privacy tools to establish the theoretical foundation. Chapter 3 presents the requirements analysis and high-level design of the tool, covering component specifications and architectural decisions with justifications. Chapter 4 details the full technical implementation of each component, including development decisions, technical challenges, and the testing strategy. Chapter 5 evaluates the tool's results against project objectives through a controlled before-and-after validation methodology. Chapter 6 addresses the legal, social, ethical, and professional issues arising from the project. Chapter 7 presents the conclusion, reflections on limitations and lessons learnt, and directions for future work.

2 LITERATURE REVIEW

2.1 Introduction

The theoretical foundation for the analysis into telemetry methods and its connotations to privacy are reviewed through two complementary lenses: software architecture explains how modern operating systems implement diagnostic data collection, while digital forensics provide methodologies for investigating the privacy implications of these systems. The combination of lenses reveals a critical gap, whilst telemetry architectures are well-documented and forensic methodologies exist for evidence analysis, no research has applied forensic principles of evidence collection and investigation to create user-accessible privacy assessment tools.

2.2 Telemetry within Modern Operating Systems

Telemetry initially began as process of error reporting, a single debugging program run by an individual user. When a user reported an error, a programmer was assigned to attach a debugger, a tool used by programmers that assist in analysing events in which a program behaviour differs from its intention (otherwise known as 'bugs', or errors) to the 'core dump', to examine and analyse the error for resolutions. This simply meant attaching a tool to analyse the current program state to deduce where the algorithm or program code deviated from the desired behaviour (Glerum et al, 2009). This method of telemetry was known as centralised logging (Riedesel, 2021), where error events were stored centrally for programmer analysis. This early model was a common way of monitoring errors that operated on a pull mechanism; Data was generated locally and transmitted only when an error occurred, giving users' implicit control over what data left their systems. As the volume of data generated started to grow exponentially, challenges arose in withstanding the millions of error reports, so the need for a new system become crucial. As a result, the shift to distributed architectures altered this dynamic, transitioning from reactive error reporting to continuous data collection. This subsequent shift to a proactive and continuous data collection left with profound privacy implicants that remain poorly understood by end users.

Distributed systems, the modern architecture for operating systems, rely on microservices to monitor data transmission from thousands of users for efficient debugging. In research conducted by Talaver and Vakaliuk (2024), Principal Component Analysis efficiently filtered anomalies across thousands of telemetry signals in distributed systems. Microsoft collects logs for dynamic analysis across millions of systems through event trace logs (ETL), the standard continuous logging tool that has replaced manual

log verification. These event logs are immutable and generated when a system executes corresponding logging code, facilitating analysis of system behaviour (He et al., 2022). Users cannot access these logs directly as they are stored as non-human-readable binary files. While these architectural advances enable system reliability, they introduce a critical asymmetry: the same infrastructure that allows Microsoft to detect anomalies across millions of systems creates a persistent, centralised surveillance infrastructure. ETL files' immutability, designed as a feature for debugging integrity, becomes a privacy liability when logs contain user behavioural data that cannot be retroactively redacted or corrected (He et al., 2022).

The Windows Registry is a further method Microsoft uses to store system and user information. Carvey (2011) establishes that the Windows Registry is not a single file but a collection of binary hives and transaction logs containing high-value metadata, a "digital diary" of user interactions, including which folders a user opened, what applications were used, and view preferences. The Registry controls the diagnostic data level determining the volume of data stored locally and transmitted to Microsoft, along with metadata for services responsible for telemetry transmission such as DiagTrack, the primary service responsible for transmitting system information to Microsoft endpoints, in conjunction with the Customer Experience Improvement Program (Radar, 2022). All of these data collection mechanisms constitute forensic artefacts that can be collated as evidence for a privacy assessment, as they are behavioural indicators that identify the user and their interactions with the system, an issue this project directly addresses.

2.3 Privacy Concerns and Ethical Implications

Privacy exposure has become a pivotal topic of debate within the technology industry, particularly where its implications emerge from forensic examination of diagnostic artefacts. The types of information stored in the Windows Registry vary from system information and software details, to user activity and security information. Detmering and Ray (2025) identify the path:
"C:\Windows\System32\Config\SYSTEM" as particularly significant, recording system configuration settings including Windows services, active background applications, and network settings. Diagnostic data also persists within event log files after a user session ends: deleted file references, access times, and program execution logs remain as forensic artefacts. Without forensic tools to surface these artefacts, users remain unaware of the privacy evidence their systems accumulate.

According to Maier and Harr (2020), Microsoft employs dark design patterns in its privacy interface: privacy settings are pre-selected to maximum data collection, and during initial operating system setup users are forced to accept broad terms to use Windows at all. For most users, this imposed acceptance creates hesitation over perceived data control. Cohen (2025) found an industry-wide misalignment between user preferences and default privacy settings, with interface complexity acting as a significant

barrier from minimising privacy exposure. Microsoft's diagnostic data settings are distributed across services, Group Policy, and the registry, making it extremely challenging for non-technical users to exercise control — a phenomenon Cohen (2025) terms "configuration fragmentation." The cumulative effect of these complex mechanisms creates a privacy paradox where users express high privacy concern but continue to engage in behaviours that compromise it, primarily due to the fatigue of navigating complex settings (Acquisti and Grossklags, 2005). While existing academic frameworks such as the privacy calculus (Dienlin, 2024) document this issue at a theoretical level, they remain inaccessible to non-technical users, and Microsoft's Diagnostic Data Viewer provides data without risk interpretation.

2.4 Existing Privacy Tools

Microsoft's Diagnostic Data Viewer allows users to see telemetry events sent to Microsoft. However, researchers argue that providing a data viewer does not equal providing meaningful control, when data is presented only as jargon-heavy JSON files that are incomprehensible to average consumers who have little to no programming knowledge (Charnsethikul et al., 2025). Critical telemetry configurations remain scattered across Group Policy Editor, Registry Editor, and Services Manager. As these are tools explicitly designed for system administrators. Research by Cohen (2025) on privacy interface design found that distributing controls across multiple interfaces creates ‘configuration fragmentation’, where users falsely believe they have disabled data collection when system-level telemetry persists unchanged.

In response to Microsoft's inadequate privacy controls, third-party tools including O&O ShutUp10 (O&O Software GmbH, 2025), W10Privacy (Schuster, 2026), and Windows Privacy Dashboard have emerged. These tools share a common paradigm: they provide toggle switches or one-click solutions to disable telemetry settings by modifying registry keys, stopping services, and applying group policies. For instance, O&O ShutUp10 offers over 130 privacy-related settings organised into categories. However, these tools prioritise modification over assessment, violating the forensic principle of documentation before intervention (Casey, 2011). Users are presented with binary controls without first being shown their current privacy state, what data currently exists on their system, or how their exposure compares to a baseline configuration. An example is W10Privacy, which allows users to disable DiagTrack with a single click but provides no assessment of what diagnostic data has already been collected, or what PII exists within diagnostic files. Furthermore, these tools provide no quantified privacy metrics as users receive recommendations without context about severity, data volume, or risk prioritisation.

2.5 Conclusion

This literature review establishes that a critical methodological gap exists across three interconnected themes. First, the evolution of Windows telemetry from reactive reporting to proactive continuous collection creates a persistent surveillance infrastructure with forensic artefacts that users cannot interpret due to technical complexity. The transition from centralised logging to distributed systems with ETL files fundamentally altered the privacy equation: data that was once pulled only on user-initiated error reports is now pushed continuously, accumulating user behavioural data that cannot be retroactively redacted (He et al., 2022). Second, privacy exposure manifests through three forensic evidence types: configuration artefacts (registry keys determining telemetry permission levels), behavioural artefacts (running services confirming active data collection in real-time), and content artefacts (diagnostic files containing PII) that persist even when users modify surface-level settings, due to configuration fragmentation and Group Policy overrides (Carvey, 2011; Cohen, 2025). The privacy paradox (Acquisti and Grossklags, 2005) explains why users continue using Windows despite privacy concerns, not because they do not value privacy, but because dark design patterns (Maier and Harr, 2020) and technical complexity create a technical barrier. Third, existing tools fail to bridge these three evidence types into a unified assessment: Microsoft's Diagnostic Data Viewer offers transparency without interpretation, while third-party tools provide modification without prior assessment, violating forensic principles of documentation before intervention (Casey, 2011). No accessible tool applies systematic forensic evidence collection across all three artefact types to produce user-friendly, quantified privacy risk metrics. This project bridges this gap by automating forensic examination of all three evidence types and translating the results into a 0–100 exposure score that non-technical users can interpret and act upon.

3 REQUIREMENTS ANALYSIS AND DESIGN

3.1 Introduction

This chapter presents the design of the windows telemetry assessment tool, detailing the architectural and component decisions with the project objectives into a functioning system. Drawing on the forensic investigative methodology from the literature review, this design chapter emphasises the transparent assessment of privacy exposure through systematic examination of the registry configuration, service behaviour, and diagnostic file content. The chapter progresses from high-level system architecture decisions, to detailed component design and a final justification of key decisions with diagrams for visual representation of component functionality and data flow.

3.2 Requirements Analysis

3.2.1 Design Requirements

Requirements for this tool were derived through systematic analysis of academic literature on privacy assessment methodologies, forensic investigation principles and gap analysis of existing privacy tools. It was discussed in the literature review that current privacy tools such as Microsoft's Diagnostic Data Viewer and O&O's ShutUp10 only modify settings without prior assessment of the system's current configuration. This gap analysis directly informed the core functional requirement, the windows telemetry assessment tool must assess privacy exposure and apply forensic evidence collection principles which is analysis and preservation of evidence, not modification. Forensic methodology discovered by researchers like Casey (2011) and Carvey (2011) established the requirement for multi-source evidence collection, and this mandates the examination of three evidence types within this tool which are: configuration evidence from the Windows registry, behavioural evidence from Windows services, and content evidence from diagnostic data files. This approach directly translated into three designed scanner components of the tool, the registry scanner, services scanner and diagnostic data file scanner. Analysis of existing privacy tools also called for another core requirement, the addition of quantified metrics with a weighted exposure score as this enables users to understand the risk severity rather than a simple 'privacy safe' versus 'exposed' categorisations. The highly technical interpretation of Microsoft's Diagnostic Data Viewer defined the non-functional requirement for user-friendly presentation with progressive disclosure, and a report detailing the summary scores and the privacy findings.

3.2.2 Assumptions and Constraints

This methodology is only limited to Windows users as that is the chosen operating system for this tool and topic of research, reducing the accessibility for Linux users (and other related operating systems) for their own privacy assessments. Due to the intrusive nature of the features of this tool, these tools are usually restricted to users with escalated administrative privileges. For the most optimal results, users will have to escalate their user privileges, to run the tool in administrator mode. Furthermore, it must be taken into account that the user's default file system settings may limit diagnostic file access, hindering the opportunity for a comprehensive analysis of their privacy assessment. For example, the tool would assume a standard Windows installation path of 'C:\Windows\, C:\ProgramData' as customised installations or non-English version of Windows may have different paths, potentially affecting file discovery. Finally, this tool ultimately assumes that users running the tool desire a privacy assessment. Some users may have legitimate reasons for maximum telemetry, where a high exposure score would be deemed acceptable.

3.2.3 Functional Requirements

Initial objectives from the project proposal identified the need for privacy exposure quantification. The following functional requirements were identified using the MoSCow method (must have, should have, could have, won't have), this prioritisation ensured development focused on core forensic assessment capabilities while identifying future enhancement opportunities. Through iterative refinement of the literature review, these objectives translated into specific functional must have requirements: registry scanning, service enumeration, PII (personally identifiable information) from diagnostic data files, and weighted scoring. These features are core forensic evidence types that enable for a comprehensive demonstration of a privacy assessment, and allows for quantifiable evidence. The following table categorises each priority following the MoSCoW framework:

Table 1: Functional Requirements with MoSCoW Prioritisation

Priority	Requirement	Description	Rationale
Must Have	Registry Scanning	Scans specific registry paths and keys responsible for system telemetry. Read-only operation.	Core forensic evidence for monitoring telemetry settings per Casey (2011) and Carvey (2011).
Must Have	Service Enumeration	Identifies active Windows services sending PII to Microsoft endpoints.	Essential for mapping data transmission channels.

Must Have	Diagnostic Data File Analyser	Enumerates PII within pre-listed diagnostic folder paths.	Locates stored sensitive diagnostic data.
Must Have	Weighted Scoring Algorithm	A universal calculator that aggregates scan results into an overall exposure score.	Enables quantifiable privacy assessment.
Must Have	Report Generation	Creates a report after each scan (individual or comprehensive).	Ensures results are comprehensible for the user.
Must Have	Comprehensive Scan	A single function that executes all scanning features at once.	Provides a total system privacy overview.
Should Have	Individual Scan Modes	Allows users to run core scanners independently rather than a full scan.	Offers flexibility and reduces time for targeted investigations.
Should Have	Export Function	Saves results to a text file for external documentation.	Supports forensic principles of evidence preservation (Casey, 2011).
Could Have	Historical Comparison	Stores results to show privacy exposure trends over time.	Demonstrates the impact of system changes or updates.
Won’t Have	Automated Remediation	Directly altering or modifying system settings.	Avoids redundancy with existing tools like O&O ShutUp10, maintaining focus on assessment.

3.2.4 Non-functional Requirements

This section will follow the same protocol by using the MoSCoW frameworks when defining the non-functional requirements for this telemetry assessment tool, addressed in the following table:

Table 2: Non-functional Requirements MoSCoW Prioritisation

Priority	Requirement Name	Description	Rationale
Must Have	Forensic Integrity	Operations must be strictly read-only in all features.	Abides by digital forensic methodology to maintain the integrity of the system's state for evidence gathering.
Must Have	Scanning Performance	Comprehensive scans must be completed within two minutes on standard hardware.	Ensures high user engagement and prevents the tool from being perceived as a burden on system resources.
Must Have	User-friendly GUI	Application must include a user interface, a tabbed GUI navigation system.	Condenses complex results into a manageable format that allows users to navigate features individually.
Must Have	Accessibility	Reports must be written for non-technical users, avoiding technical keyword overload.	Ensures that the privacy assessment is comprehensive to the average user and not just for forensic experts.
Should Have	Privilege Verification	A function to check if the tool is running with administrative privileges.	Administrative rights are a functional dependency for accessing deep-level telemetry and registry data.

Could Have	Cross-Platform Compatibility	Future exploration of privacy issues within Linux and macOS environments.	Would allow for a broader investigation into privacy exposure trends across different operating systems.
Won’t Have	Multi-OS Support	This version will not support non-Windows operating systems.	The tool relies on Windows-specific APIs, and cross-platform support would require a complete architectural redesign.

3.3 Design

3.3.1 System Architecture

This tool follows a three-layered architecture software design, consisting of organised layers each performing a specific role. The presentation layer is the topmost layer that users interact with directly, represented by the tabbed GUI layout. The business logic layer includes the scanner components and the exposure score calculator, the "heart" of the application with core logic and services (Dhiman, 2025). The data access layer is responsible for communication between the business logic components and the relevant APIs: registry access, service queries, and file system operations. This modular approach enables individual testing of each component through clear separation of the GUI from the business logic, and most importantly aligns with forensic methodology by enforcing the principle of evidence collection and analysis as distinct, auditable stages.

Figure 1 – UML Diagram of Three-Layered Architectural Design

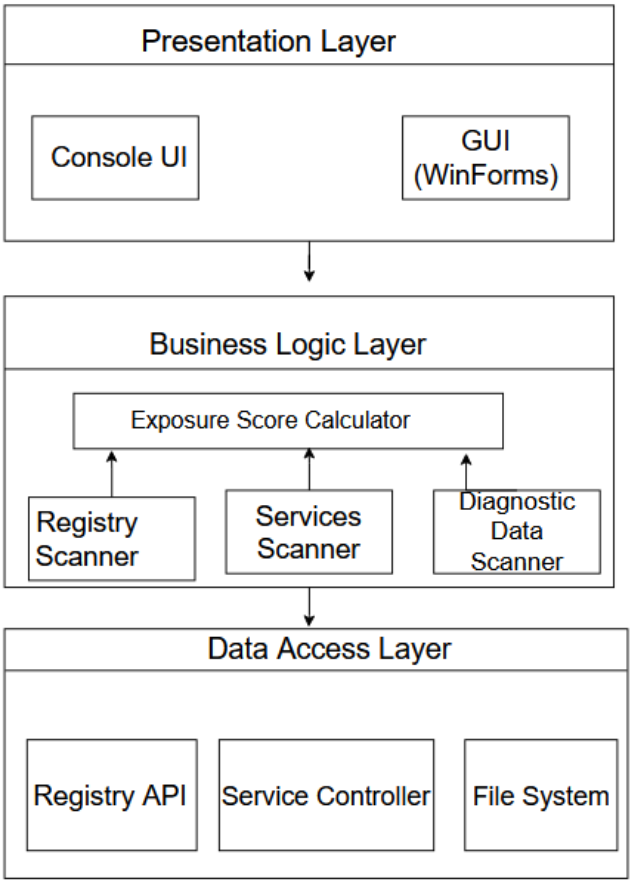


Figure (1) shows the three-tier layered architecture, indicating the separation between the presentation (UI), the business logic layer with the scanner features and the exposure score calculator, and the data access layer which includes the Windows APIs, with the arrows highlighting the dependency flow.

Alternative architectures were considered and rejected. Monolithic architecture was rejected due to inability to independently test scanner components without full GUI instantiation. Microservices architecture was rejected as unnecessary for a local application with no distribution requirements, where network communication would introduce unacceptable latency. Details of both alternative designs are provided in Appendix B. The three-tier layered architecture therefore provides the optimal balance for the stated requirements.

3.3.2 Component Overview

The tool comprises of five major components: registry scanner, services scanner, diagnostic data analyser, exposure score calculator and the user interface. This section describes component responsibilities, interfaces, and its design choices (with implementation details deferred to Chapter 5).

3.3.3 Component Design

This section will cover the internal design of each major component with technical detail, and justification for the chosen implementation. The following diagram demonstrates the previously mentioned three-layered software architecture design applied to the design of this tool's five components:

Figure 2: Component Diagram

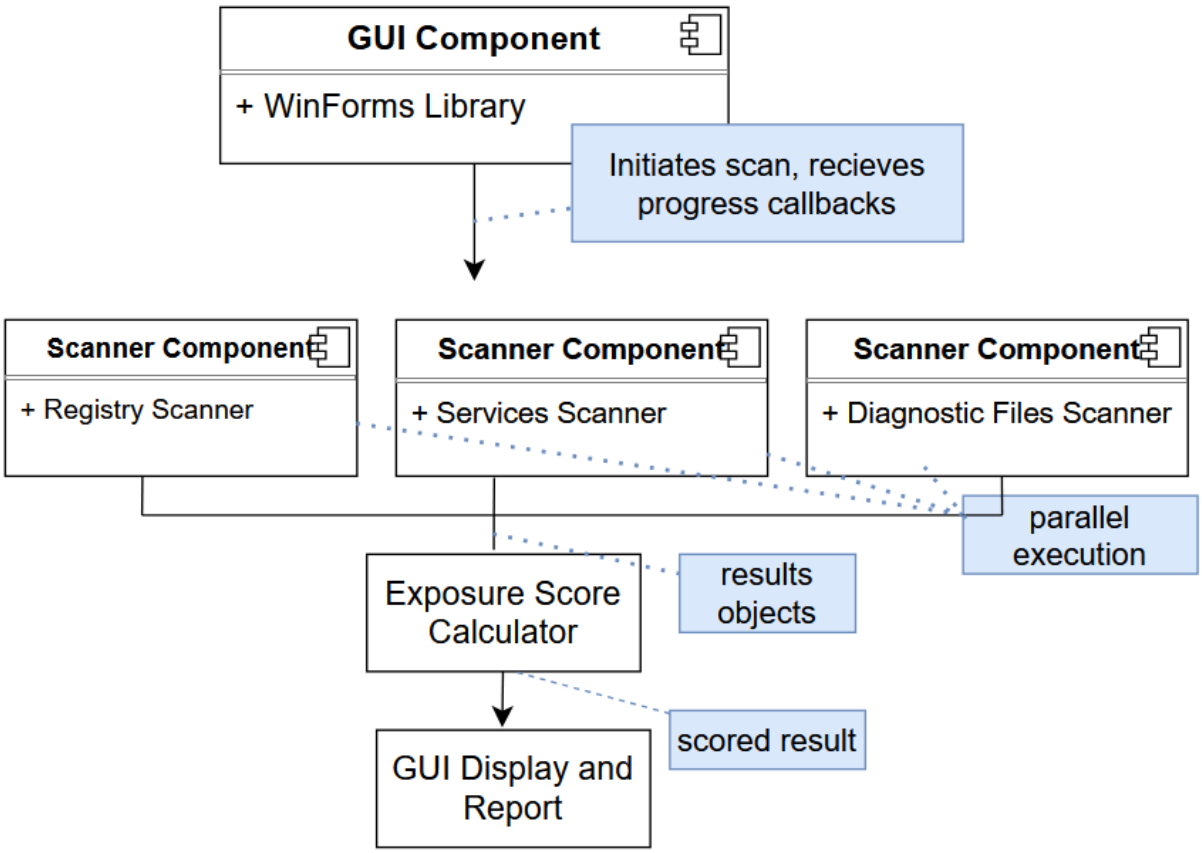


Figure 2 highlights the dependencies of each component with arrows, and the highlighted dashed arrows explaining the purpose of each stage. This diagram serves to visualise the overall design of the entire tool, with the upcoming sections detailing the design for each component.

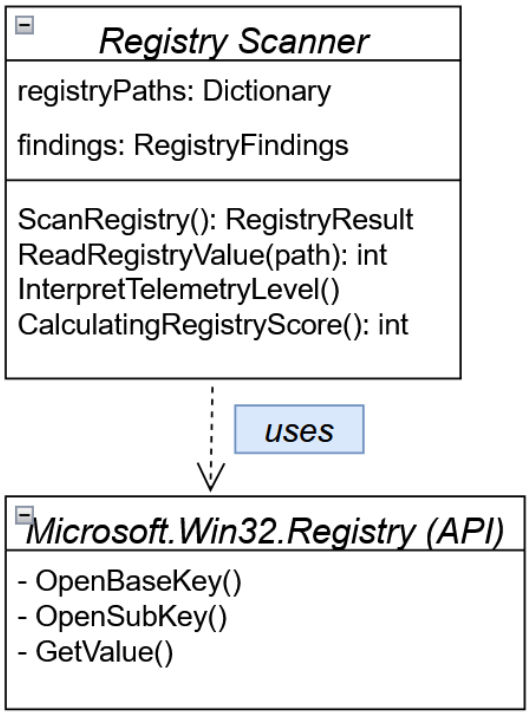
3.3.4 Registry Scanner Component

The registry scanner extracts configuration evidence from the Windows Registry, targeting two specific paths rather than conducting a broad scan. This targeted approach reflects Carvey's (2011) evidence collection principle, which prioritises reproducible results over ambiguous data collection. The two keys examined are AllowTelemetry and CEIPEnable found under the following paths:

- SOFTWARE\Policies\Microsoft\Windows\DataCollection
- SOFTWARE\Microsoft\SQMClient\Windows

These were identified as the primary configuration controls for Windows diagnostic data levels and the Customer Experience Improvement Program, confirmed through Microsoft's official diagnostic data documentation (DHB-MSFT, 2024). Their forensic significance is reinforced by the finding that configuration settings persist in the registry even when users adjust surface-level Privacy Settings through the graphical interface, due to Group Policy overrides and configuration drift (Detmering and Ray, 2025). The Microsoft Registry API was selected for this component as it provides managed, read-only access with no write operations exposed in the implementation, directly satisfying the forensic integrity non-functional requirement. The read-only constraint was a fundamental design requirement derived from the project aims in Chapter 1.2. Figure 3 highlights the relationship between the registry scanner and the Windows Registry API, with the dashed arrow demonstrating the dependency. This component directly fulfils Functional Requirement: Registry Scanning (Must Have).

Figure 3 – Registry Scanner API Class Diagram

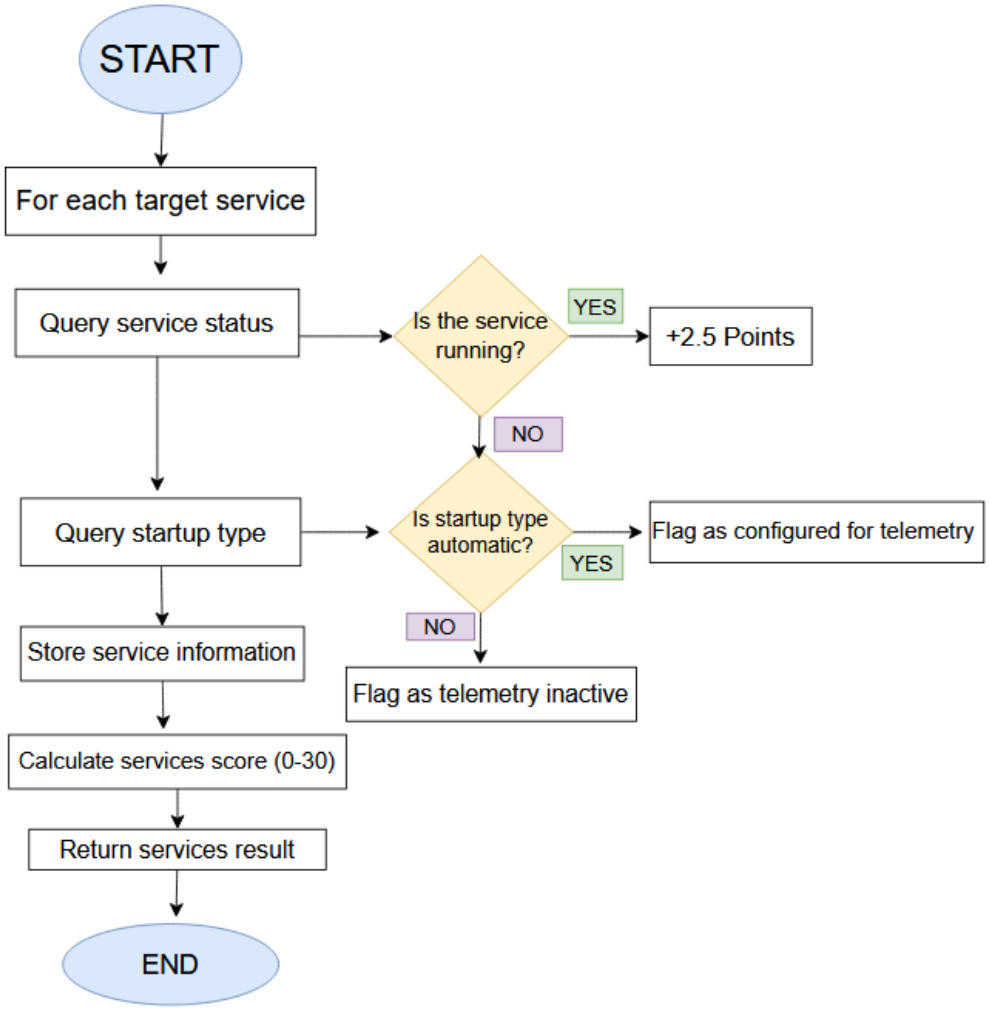


The GetValue() method, gathered from the documentation (dotnet-bot 2026), ensures that during development no write operations are exposed.

3.3.5 Services Scanner Component

The services scanner produces behavioural evidence by enumerating the runtime state and startup configuration of 12 telemetry-related Windows services. The distinction between configuration evidence from the registry and behavioural evidence from services was a deliberate design decision grounded in forensic methodology: the registry determines what telemetry is permitted, while active services confirm that data collection is occurring in real-time. The twelve services span distinct categories of data collection activity: DiagTrack as the primary telemetry transmission service, WerSvc as the historical error reporting service, and others including PcaSvc and InventoryCollectorSvc tracking application usage and hardware patterns. This multi-service scope was necessary to satisfy Objective 1's requirement for multi-source forensic evidence collection, since solely relying on the status of DiagTrack would underrepresent the full scope of active data transmission. The component was designed to query two service attributes: runtime status (running, stopped, or paused) and startup type (automatic, manual, or disabled). Documenting both provides forensic context that a single status query cannot, allowing for a comprehensive report analysis. This component directly fulfils Functional Requirement: Service Enumeration (Must Have).

Figure 4 – Services Scanner Component Design Flowchart



Each service is weighted equally at 2.5 points per running service, allowing for a total of 30 points overall that would contribute towards the exposure score (exposure score being weighted out of 100 points). This linear weighting was implemented to reflect that each service represents an individual telemetry method, and so running multiple services contributes to a more privacy exposure. Running services are those with the status of 'ServiceControllerStatus.Running', these are the services that contribute to the exposure score. Stopped, paused or inaccessible services do not accumulate any points.

3.3.6 Diagnostic Data Analyser Component

The diagnostic data analyser produces content evidence by enumerating PII within diagnostic file storage locations across the local Windows installation, directly fulfilling Functional Requirement: Diagnostic Data File Analyser (Must Have). This component addresses the core research gap articulated in the literature review: no existing tool surfaces PII already accumulated within the user's own system in a format accessible to non-technical users. Microsoft's Diagnostic Data Viewer exposes this data as incomprehensible JSON logs, while third-party modification tools make no attempt to assess existing stored content before recommending or applying changes.

Ten diagnostic directory paths were selected as scan targets, identified through forensic analysis of Windows diagnostic data documentation and Carvey's (2011) methodology. These paths represent distinct categories of diagnostic artefacts capturing continuous telemetry, and are explained in detail within Appendix B. Broad file system scanning was deliberately excluded to uphold the data minimisation principle discussed in Chapter 6 and to satisfy the two-minute performance non-functional requirement. PII detection is implemented through regex pattern matching across three categories: email addresses, IP addresses, and Windows file paths containing usernames. These categories were selected based on Microsoft's own documentation (Microsoft, 2024) and Detmering and Ray's (2025) forensic sensitivity analysis. The regex approach was chosen because it produces deterministic and reproducible results, a fundamental requirement for forensic evidence that must be consistently replicable and auditable. Machine-learning-based detection would introduce probabilistic results inconsistent with forensic standards. The known limitation is that binary ETL files cannot be inspected through regex, requiring specialist parsing tools beyond this project's current scope. This component is the highest-weighted forensic evidence category at 40 points, reflecting the evidence hierarchy where confirmed stored PII is a more direct demonstration of privacy exposure than configuration or behavioural evidence.

3.4 Conclusions

This chapter presented the requirement analysis and high-level design of the Windows Telemetry Assessment Tool. Requirements were systematically derived from literature review findings and then prioritised using MoSCoW method to establish a clear development scope. The design employs a three-layered architecture enabling testing of individual components (the registry scanner, services scanner, diagnostic file analyser, exposure score calculator, and GUI). Chapter 4 details the implementation of this design, describing the architectural blueprints and component specifications that translated into a functioning .NET application, including the technical challenges encountered and solutions applied.

4 IMPLEMENTATION

4.1 Introduction

This chapter details the development implementation of the Windows Telemetry Assessment Tool, translating the forensic-based design presented in the Design chapter into a functioning .NET application. The implementation follows a modular development approach, building and testing each component individually before integration into the complete system.

4.2 Development Environment and Technical Setup

The tool was developed using Microsoft Visual Studio 2026 as the integrated development environment, with native support for .NET frameworks including debugging tools, APIs, and NuGet package management for accessing native Windows features. The initial development used the .NET 10.0 SDK framework, which provides long-term support through 2026 with native Windows API wrappers including 'Microsoft.Win32' and 'System.ServiceProcess'. Git version control with a dedicated GitHub repository was maintained throughout development, with daily commits during active phases to demonstrate incremental progress and provide rollback capability when implementation decisions changed. Visual Studio's built-in MSTest framework was used for integrated test execution with code coverage analysis and mock support for Windows API dependencies.

The integrated dependencies were limited to Windows-native APIs, 'Microsoft.Win32.Registry' for registry access, 'System.ServiceProcess' for service enumeration, 'System.IO' for file system operations, and 'System.Windows.Forms' for GUI implementation, with no third-party NuGet packages, minimising deployment complexity and security surface area.

4.3 Component Implementation

This section will explain the technical implementation of each major component designed within Design chapter. This will include the technical implementation of the registry scanner, services enumeration, diagnostic file analyser and the exposure score algorithm.

4.3.1 Registry Scanner Implementation

This registry scanner is responsible for extracting configuration evidence from the Windows Registry. This class worked by scanning the user's Windows registry against a predefined filter of specific software in specified registry paths. This is a read only operation to prevent overwriting the services as this could severely disrupt system functionality. The scanner uses Microsoft.Win32.Registry API specifically the OpenBaseKey() method to access HKEY_LOCAL_MACHINE with RegistryView.Registry64 to ensure 64-bit registry access on x64 systems. Following is a code snippet of the API implementation:

```csharp RegistryKey baseKey = RegistryKey.OpenBaseKey(RegistryHive.LocalMachine, RegistryView.Registry64); ```

Two critical registry keys were being examined:

- 'SOFTWARE\Policies\Microsoft\Windows\DataCollection\AllowTelemetry'
- 'SOFTWARE\Microsoft\SQMClient\Windows\CEIPEnable'

The 'GetValue()' method is used for all reads, ensuring no write operations are exposed during development. AllowTelemetry values are interpreted according to Microsoft's documentation (DHB-MSFT, 2025): 0 = Security (0 points), 1 = Basic (8 points), 2 = Enhanced (17 points), 3 = Full (25 points). CEIPEnable contributes 0 or 5 points as a binary value, giving a maximum registry sub-score of 30. This proportional mapping ensures the score reflects the graduated nature of telemetry permission levels rather than treating all non-zero values as equivalent.

Defensive programming was implemented to handle missing or inaccessible registry keys. The implementation attempts to open each subkey path sequentially, catching every 'SecurityException' and 'UnauthorizedAccessException'. Rather than throwing exceptions that halt execution, the scanner returns a 'RegistryResult' object with an 'IsAccessDenied' flag, preserving partial results for forensic documentation. Missing keys are treated as an Enhanced telemetry level (value 2) for scoring purposes, consistent with Windows default behaviour, and documented in the results with a 'KeyMissing=true' flag.

4.3.2 Technical Challenges

Initial implementation used 'RegistryView.Default' which causes inconsistent results on 64-bit systems due to registry redirection. Testing on Windows 11 revealed that AllowTelemetry key reads returned null despite the key existing in the 64-bit registry hive. The solution to this challenge was to explicitly specify RegistryView.Registry64 for all registry operations.

4.3.3 Services Scanner Implementation

The telemetry services scanner enumerates Windows service states using the System.ServiceProcess.ServiceController class. Implementation required careful handling of service name variability and access permission restrictions. The enumeration strategy was to target a predefined list of 12 telemetry related services detailed in the Design chapter, and for each target service the scanner attempts to instantiate a 'ServiceController' object and query its status and start type properties, the behavioural evidence. As some services use session dependant naming such as the OneSyncSvc service, the implementation employs ServiceController.GetServices() to retrieve all services the filters using string pattern matching:

```
‘ csharp var allServices = ServiceController.GetServices(); var oneSyncServices = allServices.Where(s
        => s.ServiceName.StartsWith("OneSyncSvc", StringComparison.OrdinalIgnoreCase) ); ‘
```

This approach ensures detection regardless of session ID variations across different Windows installations. Querying service status requires user permissions but querying service configuration (the start type) requires elevated privileges on some services, so the scanner implements a tiered access strategy:

1. Attempt full query (Status and Start type)
2. If 'InvalidOperationException' is thrown, retry with status only query
3. Mark the service result with 'PartialInformation=true' if start type is unavailable

This approach ensures correct detection regardless of session ID variations across different Windows installations. Querying service configuration (startup type) requires elevated privileges on certain services, so the scanner implements a tiered access strategy: first attempting a full query for both status and startup type; if 'InvalidOperationException' is thrown, retrying with a status-only query and marking the result with 'PartialInformation=true'. This ensures the scan completes with maximum available information given the current privilege level, while logging which services require administrator privileges for complete results.

The service status and startup type combination provides forensic context within the matrix in **Table 3**. This matrix informs the scoring algorithm: an Automatic startup with Running status contributes the maximum exposure points, while Disabled with Stopped contributes zero. Startup type is queried and documented for forensic reporting but does not directly modify the exposure score: a service configured as Automatic but currently stopped represents currently reduced exposure (possibly due to user intervention), so the score reflects current runtime state rather than potential future state.

Table 3 – Service Startup & Status Matrix

Startup Type	Status	Forensic Interpretation
Automatic	Running	Full telemetry active, this status is the highest privacy exposure
Automatic	Stopped	User intervention likely due to a manual stop, a temporary privacy gain
Manual	Running	Triggered data collection is active; the trigger source would require investigation
Manual	Stopped	Reduced telemetry, as the data collection only occurs on an explicit trigger
Disabled	Stopped	Telemetry capability is removed; this is the optimal privacy state

The scanner returns a 'ServicesResult' containing a collection of 'ServiceInfo' objects per target service. The scoring algorithm iterates this collection and contributes 2.5 points per service with status 'ServiceControllerStatus.Running', with a maximum of 30 points for all twelve services running. The complete list of twelve monitored services with their functions and forensic justifications is provided in Appendix B.

4.3.4 Technical Challenge

On Windows 11 systems with restricted telemetry configurations, certain services such as DiagTrack may not exist at all rather than existing in a Disabled state. Initial implementation threw 'ServiceNotFoundException', halting the scan. Wrapping 'ServiceController' instantiation in a try-catch and treating non-existent services as not present (equivalent to Disabled for scoring) resolved this without compromising forensic accuracy.

4.3.5 Diagnostic Data Analyser Implementation

The diagnostic data analyser implements forensic examination of diagnostic file storage locations, detecting PII through regex pattern matching. This component required the most complex implementation due to file system permissions, diverse file formats, and performance optimisation requirements.

The analyser scans ten directory paths, each targeting a distinct category of diagnostic data. The folder enumeration strategy uses 'System.IO.Directory.GetFiles()' with 'SearchOption.AllDirectories' for recursive enumeration and 'File.ReadAllText()' for reading plaintext content. All file access operations are wrapped in 'try-catch' blocks for 'UnauthorizedAccessException' and 'IOException', producing structured error results for inaccessible directories rather than halting the scan - consistent with the defensive programming pattern applied throughout all scanner components and the forensic principle of documenting what was attempted and why access was restricted. The full list of ten diagnostic directory paths with their forensic justifications is provided in Appendix B.

For files that are accessible and human-readable (stored as plaintext, XML, or JSON) rather than binary, the analyser applies regex pattern matching to three PII categories. **Table 4** presents the patterns and forensic justification for each.

Table 4: PII Detection Categories using Regex Patterns and Forensic Justification

PII Type	Regex Pattern	Forensic Justification
Email Address	[A-Za-z0-9 ._%+-]+@(A-Za-z0-9 .-]+\.[A-Z | a-z]{2}	Email addresses are a primary user identifier, explicitly covered by the GDPR Article 4(1) as personal data (GDPR, 2013). Microsoft's account-linked telemetry can embed account email addresses in diagnostic log metadata (Microsoft, 2024), so their presence in a diagnostic file directly indicates identity-linked data collection.
Windows File Paths containing Usernames	C:\\Users\\[^\\]+\\	File path strings within diagnostic logs continuously contain the Windows username, a direct identifier, also

		revealing user account names and document organisation patterns. The document organisation patterns such as document naming conventions may expose both identify and document subject.
IP Addresses	\b(?:\d{1,3}\.{3}\d{1,3}\b	IP Addresses are considered personal data when linked to individuals (GDPR Recital 30) which can infer geographical location and network identity. Carvey (2011) notes network configuration data as high-value forensic artefacts and IP Address logging is present within network diagnostic logs and WMI event logs.

The regex approach was selected over machine-learning-based detection because it produces deterministic and auditable results, a fundamental forensic requirement. Detection patterns apply to .log, .txt, .xml, and .wer file types. Binary ETL files are logged as scanned but inaccessible, preserving forensic documentation of their existence. Progress reporting is implemented via a callback delegate passed from the GUI layer using 'Control.InvokeRequired' to invoke 'UpdateProgress()' on the UI thread, satisfying the non-functional requirement for a responsive interface during long-running scans.

4.3.6 Exposure Score Calculator Implementation

The exposure score calculator integrates raw outputs from all three scanners into a single 0–100 privacy exposure score. It is implemented as a dedicated class within the business logic layer, receiving typed result objects from each scanner and returning an `ExposureScoreResult` object. The calculator contains no file system, registry, or service access of its own, maintaining a clean separation from the data access layer and ensuring the scoring logic is independently unit-testable. **Table 5** summarises the weighting rationale.

Table 5: Exposure Score Weighting Rationale

Component	Evidence Type	Max Score	Justification
Registry Scanner	Configuration	30 Points *Registry Score = AllowTelemetry Points + CEIPEnable Points (Max 30 points)*	Registry configuration determines the existence of data collection through configuration settings but does not confirm whether the data has been transmitted or stored. It is therefore weighted as a secondary indicator of potential exposure rather than confirmed exposure.
Services Scanner	Behavioural	30 Points *Services Score = Number of Running Services x 2.5 (Max 30 points)*	Active services confirm that telemetry data collection is occurring in real-time, representing the current and ongoing exposure. This is a strong indicator than configuration alone but does not confirm what data has been retained, hence equal weighting with the registry component.
Diagnostic Data Analyser	Content	40 Points *Diagnostic Data Score = Data Volume Score + PII Detection Score (Capped at 20 points) + ETL Count Score (Max 40 Points)*	Actual PII is detected within diagnostic files which makes this the most direct forensic evidence of privacy exposure. Unlike configuration or

			service state, the presence of PII in a file is confirmed retained data, representing the highest severity evidence and therefore the highest weighting in the algorithm.

To maintain transparency in the scoring process and to support the forensic principle of auditable evidence, the 'ExposureScoreResult' class implements a 'GetScoreBreakdown()' method that returns a human-readable list of the component scores and total result. This result is rendered within the GUI results display and is also included in the generated report export ensuring users can verify how their total score was derived. The output takes the following form:

Figure 5: Exposure Score Output

```
Registry Configuration: [Score]/30 points
Active Services: [Score]/30 points
Diagnostic Data: [Score]/40 points
Total Exposure Score: [Score]/100
Exposure Level: [Minimal | Low | Medium | High]
```

This breakdown satisfies both the functional requirement for transparent, quantified metrics and the non-functional requirement for a user-friendly presentation that supports informed decision-making rather than alerting users to a pass or fail state.

4.4 Testing

The testing strategy for this tool was structured around three categories corresponding to the tool's modular architecture: unit testing of each scanner component, integration testing of the combined scans and the scoring algorithm, and forensic accuracy validation against native Windows tool. Visual Studio's MSTest framework was used for unit and integration testing, providing code coverage analysis and mock support for Windows API dependencies.

4.4.1 Unit Testing

Unit tests were written for each scanner component independently. For the registry scanner, tests validated that AllowTelemetry values 0–3 mapped correctly to their respective point contributions (0, 8, 17, and 25 points), that missing registry keys returned 'KeyMissing=true' without exception, and that inaccessible keys returned 'IsAccessDenied=true'. Mock registry keys were used to simulate each value without requiring a live registry, allowing tests to be executed in isolation. For the services scanner, tests verified that 'ServiceNotFoundException' was caught and treated as a non-existent service contributing 0 points, that the tiered access strategy correctly fell back to status-only queries when startup type access was denied, and that OneSyncSvc pattern matching correctly detected service name strings with and without session ID suffixes. For the diagnostic data analyser, unit tests validated the three regex patterns against known test strings, confirming correct detection and reviewing false positive behaviour against strings resembling but not constituting PII.

4.4.2 Integration Testing

Integration testing verified that the exposure score calculator correctly aggregated results from all three scanner components into the expected total score. Test scenarios included a maximum configuration which was confirmed to produce 100 points, and a minimum configuration which was confirmed to produce 0 points. This included setting all registry keys to the max value, all twelve services running, and enabling the maximum PII instances, as for the minimum configuration this involved setting registry keys to security level, with no running services, and no present diagnostic files. The GetScoreBreakdown() method was verified to return each correctly formatted component scores matching the sub-totals passed by each scanner. The comprehensive scan function was tested to confirm that all the three scanners executed in the correct sequence, that results were passed correctly to the calculator, and that the GUI updated with real-time progress callbacks at each stage. Scan completion time was measured across multiple executions on the test device, consistently returning within the two-minute non-functional performance requirement.

4.4.3 Forensic Accuracy Validation

Forensic accuracy validation was conducted on a self-owned Windows device by comparing the tool output against manual inspection using native Windows tools. The methodology and results of this validation are presented in detail in Chapter 6, the Results and Evaluation, including the before-and-after comparison using the open-source modification script, HushWin. Registry scanner accuracy was confirmed at 100% against Regedit, and services scanner accuracy was confirmed at 100% against Services.msc. The diagnostic file analyser's output was partially validated through File Explorer file checks, and sample PII instance review, with the known IP address false positive documented in the report output.

4.5 Conclusion

Chapter 4 detailed the implementation of the Windows Telemetry Privacy Assessment Tool, translating Chapter 3 design into a functioning .NET application. All functional requirements were successfully implemented, with non-functional requirements met through iterative optimisation. Technical challenges including performance bottlenecks, permission handling and regex backtracking were resolved through timeout mechanisms and measurement-driven optimisation. Testing validated forensic accuracy and performance. Chapter 5 evaluates these results against project objectives.

5 RESULTS AND EVALUATION

5.1 Introduction

This chapter evaluates the Windows Telemetry Assessment Tool against the project objectives and design requirements established in Chapter 3. The tool was executed on a self-owned Windows device in its default configuration and subsequently re-executed following application of HushWin, an open-source telemetry disabling script available on GitHub (bitlogik, 2021). HushWin works by modifying registry keys, disabling telemetry services, and applying group policy configurations to reduce Windows diagnostic data collection. This approach enabled each scanner component's output to be compared against a known, externally induced change in system state, providing a controlled basis for assessing forensic accuracy and scoring responsiveness. Section 5.2 presents results, and Section 5.3 provides critical evaluation mapping each finding to the project objectives, together with an honest assessment of limitations.

5.2 Results

5.2.1 Pre HushWin Configuration

The tool was first executed on the test device in its default Windows configuration prior to any telemetry modification. The comprehensive scan completed successfully within the two-minute performance requirement established in the non-functional requirements. The resulting exposure score breakdown was as follows:

Table 6: Baseline Exposure Score

Component	Achieved Score	Maximum Score
Registry Scanner	10	30
Services Scanner	20	30
Diagnostic Data Analyser	9	40
Total Exposure Score	39	100
Exposure Level	Low - Medium	-

The registry score of 10 indicated that AllowTelemetry was set to Basic (value 1, contributing 8 points) with CEIPEnable active (2 points). This is consistent with a partially adjusted Windows configuration as it is below the default Enhanced setting, suggesting some prior privacy adjustments. The services score of 20 indicated eight of the twelve enumerated services in an active running state at the time of the scan. The diagnostic score of 9 reflected a low volume of text-readable diagnostic files and a small number of PII instances, partially attributable to the binary ETL limitation discussed in Section 5.3.

All three scanners completed their individual executions and returned structured results to the exposure score calculator without error. Registry values were read successfully from the 64-bit registry hive using the 'RegistryView.Registry64' specification. All queried services returned both status and startup type attributes at the privilege level of the test execution.

5.2.2 Post-HushWin Configuration

Following the baseline scan, the HushWin script was applied to the test device. After the script completed its modifications, the Windows Telemetry Assessment Tool was re-executed on the same device without any changes to the tool's configurations or codebase. The resulting exposure score breakdown was as follows:

Table 7: Post HushWin

Component	Score	Maximum
Registry Scanner	0	30
Services Scanner	5	30
Diagnostic Data Analyser	9	40
Total Exposure Score	14	100
Exposure Level	Low	-

The registry score dropped from 10 to 0, confirming HushWin had successfully set AllowTelemetry to 0 (Security level) and disabled CEIPEnable. The tool correctly detected and interpreted both changes, mapping the telemetry settings to 0 points in a manner entirely consistent with the scoring algorithm. The services score reduced from 20 to 5, indicating HushWin had stopped or disabled a subset of the previously running services. The remaining 5 points reflect two services that remained active and confirmed as critical Windows services that HushWin's scope did not disable, verified manually via Services.msc. The diagnostic score remained at 9 across both scan states. This result is consistent with expected behaviour: HushWin modifies registry settings and service states but does not delete or alter existing diagnostic files already accumulated on the device. This persistence demonstrates the forensic reality that previously collected diagnostic data remains present regardless of configuration changes.

Figure 6 – HushWin Script Execution

```
Administrator: C:\WINDOWS\System32\cmd.exe

----  HushWin v 0.1  ----
  provided by BitLogiK



Disabling Logging Services ...
- Stopping Diagtrack service
- Stopping dmwappushservice service
- Stopping AutoLogger
- Stopping Diagnostics Hub Standard Collector

Disabling Telemetry Scheduled Tasks ...

Removing Compatibility Telemetry Process ...

Editing the registry ...

Performing additional actions ...
- Stopping Nvidia Telemetry
- Stopping Office Telemetry
- Disabling Remote Assistance
- Disabling Windows Media Player usage tracking

HushWin script successfully executed.
Your Windows is now safe about telemetry collections.

Hit any key to quit
```

Figure 6 above is a screenshot showcasing the script successfully running with verbose outputs of each modification.

5.2.3 Tool Accuracy Validation

Registry values were confirmed using Registry Editor (Regedit), with AllowTelemetry and CEIPEnable manually inspected at their key paths in both scan states. The values read by the tool matched Regedit exactly across both states, confirming 100% registry scanner accuracy. Service states were verified using Services Manager (Services.msc), with all twelve services manually checked before and after modification. The services scanner correctly identified running and stopped/disabled services in both states, with no discrepancies. The two residual running services in the post-HushWin state were confirmed as genuinely not disabled rather than representing a scanner error. Diagnostic file detection was partially verified through File Explorer and sample PII instance review, with one false positive category identified: version number strings in certain log files were detected by the IP address regex due to structural similarity to IPv4 addresses. This is explicitly documented in the generated report output.

Table 8: Functional Requirements Achieved

Requirement	Priority	Status
Registry Scanning	Must Have	Achieved
Service Enumeration	Must Have	Achieved
Diagnostic File Analyser	Must Have	Achieved – with ETL limitation
Weighted Scoring Algorithm	Must Have	Achieved
Report Generation	Must Have	Achieved
Comprehensive Scan	Must Have	Achieved
Individual Scan Modes	Should Have	Achieved
Export Function	Should Have	Achieved
Historical Comparison	Could Have	Not Implemented
Automated Remediation	Won't Have	Correctly Excluded

All 'must have' and 'should have' requirements were successfully implemented and validated through the controlled testing methodology. The 'could have' historical comparison feature was not implemented within the project timeline, and the won't have automated remediation requirement was correctly maintained throughout the development; there was no writing modification operations in any scanner component across either scan state.

5.3 Evaluation

5.3.1 Evaluation of Multi-Source Forensic Evidence Collection (Objective 1)

The baseline result produced a finding of evaluative significance independent of scoring validation. The test device presented a registry score of 10, suggesting prior privacy adjustments had reduced the AllowTelemetry level to Basic. However, the services score of 20 revealed that eight of twelve telemetry services remained actively running, demonstrating that registry-level configuration changes had not translated into corresponding reductions in active service behaviour. This divergence between configuration evidence and behavioural evidence concretely validates the configuration fragmentation phenomenon described by Cohen (2025): a tool that examined only registry settings would have underestimated the device's actual exposure by 20 points, while a tool that examined only service states would have missed the partial registry mitigation already in place. The combination of both evidence sources, alongside diagnostic content evidence, into a single quantified score provides a more accurate picture of privacy exposure than any single source alone, directly validating the multi-source forensic evidence design of Objective 1.

5.3.2 Evaluation of Quantified Risk Metrics (Objective 2)

The scoring algorithm responded proportionally to confirmed system state changes across both scan states. The total score reduction from 39 to 14, a decrease of 25 points, was produced entirely by the registry and services components responding correctly to HushWin induced changes, with the diagnostic component correctly remaining stable. The registry component's reduction from 10 to 0 directly corresponded to the verified AllowTelemetry change from value 1 to value 0 and CEIPEnable disablement, confirming the proportional scoring map accurately reflects Microsoft's severity gradations. The services component's reduction from 20 to 5 correctly reflected partial disablement: two services remaining active rather than all twelve demonstrates that the scanner scores each service's runtime state individually rather than treating disablement as binary. This validates Objective 2: the algorithm produces differentiated, proportional metrics rather than a binary safe/exposed classification.

5.3.3 Evaluation of User-Accessible Presentation (Objective 3)

The GUI's progressive disclosure architecture and plain-language report generation were technically implemented in accordance with non-functional specifications. The ‘GetScoreBreakdown()’ method renders component-level detail alongside the total score, enabling users to understand the source of their exposure rather than responding to an abstract number. However, whether these design decisions translate into genuine comprehensibility for users without a technical background remains unverified by external evidence, as no formal participant usability study was conducted. This is the most substantial gap between the project's stated objectives and the available evaluation evidence and is the primary direction for future work.

5.3.4 Evaluation of Assessment-First Methodology

No write, delete, or modification operations were present in any scanner component across either scan state. The read-only constraint was maintained throughout all development iterations and confirmed through both code review and the integration test's verification of the ‘Won't Have’ automated remediation requirement. The persistence of the diagnostic score at 9 across both scan states constitutes the most forensically significant finding of the evaluation. It demonstrates, through live device evidence, that configuration modification does not retroactively address PII accumulated in local diagnostic files. A user observing a score reduction from 39 to 14 following HushWin application might reasonably conclude their exposure had been substantially resolved; the tool's generated report makes explicit that 9 points of diagnostic exposure remain unaffected by any configuration change, reinforcing the assessment-first methodology as a necessary precondition for informed privacy management.

5.3.5 Limitation of the Accuracy Validation

The primary limitation is single-device testing, which constrains generalisation across the full range of Windows editions, hardware configurations, and prior privacy settings the tool is designed to support. A device with Optional diagnostic data enabled and a longer usage history would accumulate substantially more text-readable diagnostic content, likely producing a higher diagnostic score and more pronounced demonstration of the ETL limitation. The use of HushWin as the modification agent introduces a dependency on a third-party script; the two services remaining active in the post-modification state could not be definitively confirmed as outside HushWin's intended scope without a full audit of its source code, though manual Services.msc inspection confirmed they had not been set to Disabled status. The scoring methodology, whilst grounded in the forensic evidence hierarchy from the literature review, involves interpretive allocation decisions not uniquely determined by forensic theory. The exposure score should therefore be treated as a comparative and directional metric rather than an objectively calibrated measure of privacy risk; an interpretation the tool's report generation supports by presenting component breakdowns alongside the total score.

5.4 Conclusion

The evaluation demonstrated that the Windows Telemetry Assessment Tool successfully achieves its core forensic objectives across all validated scan states. All three scanner components accurately detected system state before and after modification, the scoring algorithm responded proportionally to confirmed changes, and all Must Have and Should Have functional requirements were implemented and validated. The most significant finding, the persistence of diagnostic exposure across both scan states, validates the assessment-first design rationale with concrete device evidence. The evaluation also identified meaningful limitations in validation scope and the absence of external user testing, each of which directly informs the future work directions in Chapter 7.

6 LEGAL, SOCIAL, ETHICAL AND PROFESSIONAL ISSUES

The Windows Telemetry Assessment Tool processes personal data when detecting PII in diagnostic files, triggering GDPR Article 4(2) definitions of processing. Key compliance measures were implemented in accordance with GDPR Article 5. The tool implements data minimisation by design under Article 5(1)(c): all processing occurs locally with no data transmission or cloud integration, eliminating the risk of data breaches. Following Article 5(1)(b), PII detection within the diagnostic data analyser serves solely for the purpose of the privacy assessment report. Regex patterns match email addresses, file paths, and IP addresses only to count instances: the tool does not collect, aggregate, or repurpose detected PII. To enhance transparency under Article 12, the tool clearly documents its operation by guiding users on which files are actively scanned, producing a finalising report indicating which directories were accessed, and making the source code publicly available for expert verification.

All processed information operates under Article 6(1)(f) legitimate interests: users have a legitimate interest in auditing their own privacy exposure on their own systems, the processing is necessary for that privacy assessment, and users operate the tool voluntarily on their own devices. As users process their own data on their own devices, they inherently exercise Article 15 (right of access) and Article 20 (right to portability) through the tool.

Beyond legal compliance, an ethical responsibility to present risk accurately without inducing unnecessary anxiety was addressed through neutral framing of risk levels. Labelling an average score as "medium" rather than using alarmist language such as ‘Danger’, and by focusing on presenting facts to enable personal decision-making rather than prescribing action. This also addresses privacy nihilism: rather than presenting high scores as alarming or irreversible, the tool clearly states that the exposure score is a snapshot of the current configuration and is changeable, providing conceptual guidance on how to improve it.

The tool's requirement for administrative privileges excludes users with older hardware, users in regions with limited internet access for .NET framework downloads, and users who cannot run executables with elevated privileges, potentially exacerbating existing privacy inequities by making the assessment a privilege of technically sophisticated users. Mitigations include providing the tool as a portable executable requiring no prior installation, enabling operation without administrator privileges with explicit limitation notices, and producing comprehensive plain-language documentation after each scan.

As a computing professional in training, this project adhered to the British Computer Society Code of Conduct. The tool serves the public interest by empowering users with privacy assessment capability

that was previously inaccessible without forensic expertise. Project scope was limited to current competency levels: binary ETL parsing was excluded because thorough implementation would require expertise beyond current skills, and the open-source publication on GitHub allows contributions from the wider professional community through peer review.

7 CONCLUSION

7.1 Summary of Windows Telemetry Assessment Tool Development

This project developed the Windows Telemetry Assessment Tool, a forensic-based privacy assessment application for Windows operating systems that quantifies a user's data exposure through systematic examination of three local diagnostic artefact categories: registry configuration, active service behaviour, and diagnostic file content. The tool was developed in response to the clearly identified gap in the existing privacy tool landscape: no accessible tool applied forensic evidence collection principles to produce user-interpretable, quantified privacy risk metrics. Microsoft's Diagnostic Data Viewer provided data transparency without interpretation, presenting collected telemetry as incomprehensible JSON logs inaccessible to non-technical users. Third-party tools such as O&O ShutUp10 and W10Privacy provided modification capability without prior assessment, altering system state without first establishing what data already existed on the device in violation of the forensic principle of documentation before intervention (Casey, 2011).

The tool addressed this gap through five objectives: a multi-source forensic evidence collection architecture comprising a registry scanner, a services scanner enumerating twelve telemetry-related Windows services, and a diagnostic data analyser examining ten directory paths for PII through regex pattern matching; a weighted scoring algorithm aggregating outputs into a single 0–100 exposure score with a transparent 30/30/40 breakdown reflecting the forensic evidence hierarchy; a Windows Forms GUI presenting forensic findings in non-technical language with progressive disclosure and exportable report generation; a read-only, assessment-first methodology maintained across all components throughout development; and forensic accuracy validation through a controlled before-and-after methodology verified against native Windows tools.

7.2 Summary of Overall Outcomes and Results

The tool successfully achieved its core objectives and demonstrated forensic accuracy across all validated scan states. The controlled validation produced a baseline exposure score of 39, with a post-HushWin score of 14. Registry and services components correctly reflected the system state changes induced by the script, while the diagnostic component correctly remained stable at 9. This stability across both scan states constitutes the most significant evaluative finding: it provides live device-specific evidence that configuration modification does not retroactively address PII accumulated within local diagnostic artefacts. All Must Have and Should Have functional requirements were implemented and validated. Registry scanner accuracy was confirmed at 100% through Regedit verification, and service status accuracy was confirmed at 100% through Services.msc verification. The baseline scan

additionally produced a finding of broader forensic significance; the divergence between registry score (10) and services score (20) concretely validated the configuration fragmentation phenomenon described by Cohen (2025), demonstrating that the tool's multi-source architecture was necessary to surface this divergence that a single-source tool would have missed.

7.3 Reflection on Limitations and Challenges

Several limitations were encountered during development and evaluation that are important to acknowledge in positioning the tool's contribution honestly. The most significant technical limitation is the exclusion of binary ETL files from the diagnostic data analyser. ETL files constitute approximately 75% of the files present in the ten diagnostic directories on a typical Windows installation, and they represent the primary storage format for DiagTrack telemetry. The regex-based PII detection approach, whilst appropriate for ensuring deterministic and reproducible forensic results within text-readable files, cannot inspect binary content without specialist ETL parsing tools that were beyond the current scope. As a result, the diagnostic component systematically underestimates content evidence of privacy exposure, and the diagnostic score of 9 should be understood as a minimum confirmed exposure rather than a complete measure of accumulated PII.

Testing on a single self-owned device limits the generalisation of results across Windows editions, hardware configurations, and prior privacy settings. The use of HushWin as the modification agent introduced a dependency on a third-party script whose full scope was not formally audited, though manual verification confirmed the services scanner output was accurate in both states. The GUI and report generation were technically implemented but cannot be verified as genuinely comprehensible for non-technical users in the absence of a formal usability study - this is the most substantial gap between the project's stated objectives and the available evaluation evidence.

A key lesson from the ETL limitation was that scoping a forensic tool to deterministic, auditable methods is preferable to including higher-coverage approaches that introduce probabilistic or irreproducible results. The regex approach maintained forensic integrity even at the cost of coverage, which was the correct forensic decision, but should have been communicated more explicitly in the design requirements as a documented trade-off rather than an implicit constraint. A second lesson was that validation methodology should be designed alongside the tool from the beginning of the project. Had multi-device validation been planned from the outset, it could have been incorporated into the project timeline; instead, it was identified as a limitation only during the evaluation phase, limiting the scope of evidence available to assess the tool's generalisability. A third lesson arose from the project's divergence from the original proposal: the initial proposal included an email breach checker via the Have I Been Pwned API. Through the literature review and supervisor discussions, this feature was

deprioritised in favour of deeper forensic examination of local diagnostic artefacts, which better aligned with the forensic evidence gap identified in the research. This plan change was the correct methodological decision, but it illustrates the importance of treating the project proposal as an initial hypothesis to be refined through research rather than a fixed specification.

7.4 Future Work

The most impactful technical enhancement would be ETL file parsing capability within the diagnostic data analyser. Microsoft's `tracerpt` command-line tool could be invoked locally to convert ETL files to a readable format prior to scanning, remaining consistent with the tool's offline, read-only design principles (robinharwood, 2023). Formal user acceptance testing with representative non-technical Windows users is the most important non-technical priority. A structured usability study using task completion metrics and Likert scale responses would provide the evidence base to validate the GUI design and report language, likely revealing areas where technical terminology remains insufficiently translated into accessible language. Extending validation across Windows 10 Home, Windows 10 Pro, Windows 11 Home, and Windows 11 Pro would strengthen confidence in forensic accuracy and allow calibration of the scoring algorithm against a representative range of baseline configurations. The deferred historical comparison feature would add meaningful longitudinal capability, allowing the tool to surface whether Windows updates re-enable previously disabled services or restore modified registry values - a forensically significant behaviour that no current tool tracks automatically. Finally, cross-platform exploration on Linux and macOS, while requiring substantial architectural redesign given the tool's reliance on Windows-specific APIs, would broaden the tool's impact and enable comparative forensic analysis of telemetry practices across operating systems.

REFERENCE LIST

1. Acquisti, A. and Grossklags, J. (2005). Privacy and rationality in individual decision making. *IEEE Security and Privacy Magazine*, 3(1), pp.26–33. doi:https://doi.org/10.1109/msp.2005.22.

2. Admin_CM_1 (2024). *Iterative Development and Prototyping | Design Thinking Approach*. [online] Creating Minds | Java Training Program. Available at: https://www.thecreatingminds.com/evolution-through-iteration-a-design-thinking-approach-to-iterative-development-and-prototyping/.

3. bitlogik (2021). *GitHub - bitlogik/HushWin: Shell script to disable all Windows capabilities about telemetry and tracking*. [online] GitHub. Available at: https://github.com/bitlogik/HushWin [Accessed 21 Apr. 2026].
4. Bott, E. (2026). *I've been studying Windows telemetry for a decade - here's the only setting I turn off*. [online] ZDNET. Available at: https://www.zdnet.com/article/microsoft-spying-windows-telemetry/.

5. Cambridge Dictionary. "Telemetry." *@CambridgeWords*, 14 Feb. 2024, dictionary.cambridge.org/dictionary/english/telemetry.

6. Carvey, H. (2011). *Windows Registry Forensics*. Elsevier.

7. Casey, E. (2011). *Digital Evidence and Computer Crime*. [online] Google Books. Available at: https://books.google.co.uk/books?hl=en&lr=&id=lUnMz_WDJ8AC&oi=fnd&pg=PP1&dq=casey+2011+forensic+text&ots=aMv5FiAMYb&sig=Ph71gR43rs3VMROGQpKxdoi6U5E#v=onepage&q=casey%202011%20forensic%20text&f=false [Accessed 6 Apr. 2026].

8. Charnsethikul, P., Zunquti, A., Lucas, G. and Mirkovic, J. (2025). Navigating Social Media Privacy: Awareness, Preferences, and Discoverability. *Proceedings on Privacy Enhancing Technologies*, 2025(4), pp.620–638. doi:https://doi.org/10.56553/popets-2025-0148.

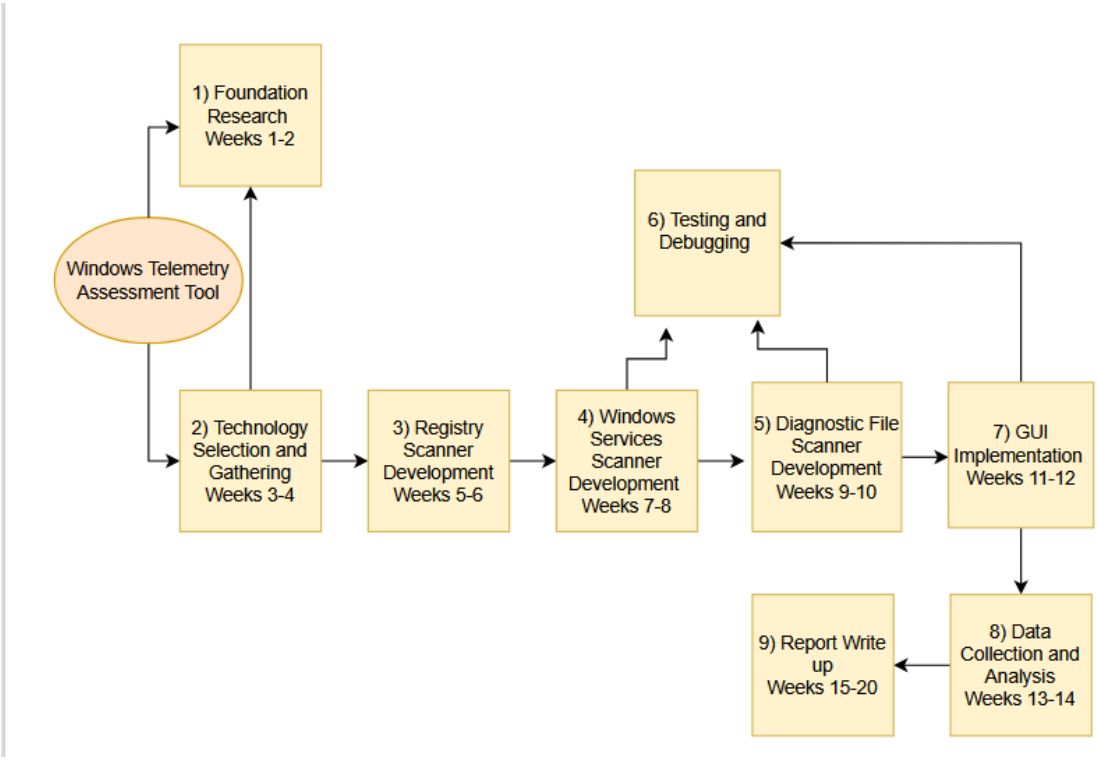
9. Cohen, J. (2025). *Click Here for Confusion: The State of Privacy Settings*. [online] Information Sciences Institute. Available at: https://www.isi.edu/news/80134/click-here-for-confusion-the-state-of-privacy-settings/.
10. Detmering, A. and Ray, C. (2025). *Windows Registry Forensics Cheat Sheet 2026*. [online] Cyber Triage. Available at: https://www.cybertriage.com/blog/windows-registry-forensics-cheat-sheet-2026/#Core%20Windows%20Registry%20Hives%20and%20Their%20Forensic%20Value [Accessed 2 Apr. 2026].
11. DHB-MSFT (2024). *Windows Privacy Compliance Guide - Windows Privacy*. [online] Microsoft.com. Available at: https://learn.microsoft.com/en-us/windows/privacy/windows-privacy-compliance-guide.
12. Dhiman, T. (2025). *What is Layered Architecture? Examples & Best Practices*. [online] Substack.com. Available at: https://imtarundhiman.substack.com/p/understanding-layered-architecture.
13. Dienlin, T. (2024). Privacy Calculus: Theory, studies, and new perspectives. *Privacy Calculus: Theory, studies, and new perspectives*. doi:https://doi.org/10.31235/osf.io/z3gw8.
14. GDPR (2013). *Art. 4 GDPR – Definitions | General Data Protection Regulation (GDPR)*. [online]
15. GDPR.EU (2018). *Recital 30 - Online identifiers for profiling and identification*. [online]
16. GDPR.eu. Available at: https://gdpr.eu/recital-30-online-identifiers-for-profiling-and-identification/.
17. General Data Protection Regulation (GDPR). Available at: https://gdpr-info.eu/art-4-gdpr/.
18. gewarren (2024). *Microsoft.Win32.Registry class - .NET*. [online] Microsoft.com. Available at: https://learn.microsoft.com/en-us/dotnet/fundamentals/runtime-libraries/microsoft-win32-registry.
19. Glerum, K.A., Kinshuman Kinshumann, Greenberg, S., Aul, G.J., Orgovan, V., Nichols, G., Grant, D.M., Loihle, G. and Hunt, G.C. (2009). Debugging in the (very) large. *Symposium on Operating Systems Principles*. doi:https://doi.org/10.1145/1629575.1629586.
20. He, S., Zhang, X., He, P., Yong, X., Li, L., Kang, Y., Ma, M., Wei, Y., Dang, Y., Saravanakumar Rajmohan and Lin, Q. (2022). An empirical study of log analysis at Microsoft. *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. doi:https://doi.org/10.1145/3540250.3558963.
21. kaushika-msft (2026). *Introduction to TroubleShootingScript toolset (TSS) - Windows Client*. [online] Microsoft.com. Available at: https://learn.microsoft.com/en-us/troubleshoot/windows-client/windows-tss/introduction-to-troubleshootingscript-toolset-tss [Accessed 21 Apr. 2026].

22. Khan, S. and Mahadik, S. (2022). *A Comparative Study of Agile and Waterfall Software Development Methodologies*, 2(1). doi:https://doi.org/10.48175.
23. Maier, M. and Harr, R. (2020). Dark Design Patterns: An End-User Perspective. *Human Technology*, 16(2), pp.170–199. doi:https://doi.org/10.17011/ht/urn.202008245641.
24. Microsoft (2024). *Data collection summary for Windows*. [online] Microsoft.com. Available at: https://www.microsoft.com/en-us/privacy/data-collection-windows.
25. Microsoft Windows 10 breaches Dutch privacy law. (2017). *BBC News*. [online] 16 Oct. Available at: https://www.bbc.co.uk/news/technology-41634617.
26. Nissenbaum, H. (2009). Privacy in Context. *Privacy in Context Technology, Policy, and the Integrity of Social Life*. doi:https://doi.org/10.1515/9780804772891.
27. O&O Software GmbH. (2025). *O&O ShutUp10++ - O&O Software GmbH*. [online] Available at: https://www.oo-software.com/en/shutup10.
28. Radar, J. (2022). *what Is DiagTrack. Can You Turn It Off? - TechRadarto*. [online] TechRadarto. Available at: https://www.techradarto.com/diagtrack/#DiagTrack_Definition [Accessed 26 Mar. 2026].
29. Riedesel, J. (2021). *Software Telemetry: Reliable logging and monitoring*. [online] Google Books. Available at: https://books.google.co.uk/books?hl=en&lr=&id=_BY6EAAAQBAJ&oi=fnd&pg=PA1&dq=telemetry+as+error+logging+&ots=btseGXZsQl&sig=q0gUhBtUfM8hxnuP6tDhyo5Xe9Q#v=onepage&q&f=false [Accessed 24 Mar. 2026].
30. robinharwood (2023). *tracerpt*. [online] Microsoft.com. Available at: https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/tracerpt [Accessed 20 Apr. 2026].
31. Schuster, B. (2026). *W10Privacy - Privacy made easy*. [online] W10Privacy. Available at: https://www.w10privacy.de/english-home [Accessed 6 Apr. 2026].
32. Talaver, O.V. and Vakaliuk, T.A. (2024). Telemetry to solve dynamic analysis of a distributed system. *Journal of edge computing*, 3(1). doi:https://doi.org/10.55056/jec.728.
33. Tan, L., Su, W., Zhang, W., Lv, J., Zhang, Z., Miao, J., Liu, X. and Li, N. (2021). In-band Network Telemetry: A Survey. *Computer Networks*, [online] 186, p.107763. doi:https://doi.org/10.1016/j.comnet.2020.107763.

APPENDIX A - Project Proposal

a.1 Revised Proposal

Revised Project Plan:



a.2 Original Proposal

UNIVERSITY OF GREENWICH

FACULTY OF ENGINEERING AND SCIENCE

SCHOOL OF COMPUTING AND MATHEMATICAL SCIENCES

COMP1682 Final Year Project Proposal

A Local Telemetry Assessment Tool:

Measuring the extent of data exposure through system telemetry

Ieva Paskeviciute
001303411
BSc (Hons) Cyber Security and Digital Forensics

Supervisor: Nowroozi, Ehsan / Waqas, Muhammad

Word count: 3251

April 2026

DECLARATION OF AI USE

Please complete this part when you have used AI during the process of undertaking this assignment to acknowledge the ways in which you have used it.

I have used AI while undertaking my assignment in the following ways:

- To develop research questions on the topic – YES
- To create an outline of the topic – YES
- To explain concepts – NO
- To support my use of language – NO
- To summarise the following articles/resources: – NO
- In other ways, as described below: – YES
  - To provide a structure template to writing out my ideas and approaches especially for section 3. Methodology and Technical Approach.

SYNOPSIS

Most modern technologies and online software rely on automatic data collection and transmission of data for monitoring, to analyse and enhance its performance. To comply, users are notified of their **privacy** rights through their **privacy** policies which is usually in the form of a checkbox for user **consent**. However, despite this, it has been proven that the proposed **privacy** policies are not disclosed thoroughly, leading to a false interpretation of user **consent**, and the collection of mass amounts of personal information from a user without their acknowledgement.

An example of a major tech corporation responsible for this that will be mentioned frequently throughout this project is Microsoft, specifically how Windows operating system monitors and stores mass amounts of personal data. One way Microsoft does this is by storing user data within its registries, and Amoruso (2024) developed an application called 'SeeShells' utilising Windows API for registry interaction to unveils user activity and user information all without triggering Windows security, ultimately proving the extent to Windows breaching user **privacy** as well as how poorly personal information is managed as the data uncovered by Amoruso was unencrypted and found in plain-text. As a result of this mass data tracking, the chances of data breaches increase exponentially, all without the user's knowledge.

This project aims to investigate the extent of personal data exposure through software telemetry, comparing with existing tools that categorises data collection practises and filtering specific software infamous for this, and build a tool that empowers users to visualise and quantify their data leakage by using scanners that filter specific Microsoft policies that send user data, and open API's software such as the website 'Have I Been Pwned' that reveal data breaches via personal emails.

Keywords: [**telemetry, privacy, consent**]

1. AIM AND OBJECTIVES

This project aims to investigate the extent of personal data exposure through software telemetry from Microsoft Windows and personal emails using open-source data breach checkers like Have I Been Pwned, and to develop a tool as a result that empowers users to visualise and quantify their own data leakage.

7.6.1 Objective 1

- Research and categorise the most commonly collected personal data by telemetry in Microsoft Windows operating system. Look into documented Microsoft Windows endpoints responsible for user data collection.
- This will be addressed in the form a table.
- This is important as it will highlight the specific telemetry to filter when developing my tool.
- This will take ~1 week.

7.6.2 Objective 2

- Research into the context and scale of known data breaches, as this is a topic my project will leverage from.
- Research into known tracking tools that provide user awareness about recent data breaches, regarding their functionality, such as the website 'Have I been pwned'.
- This will take approximately ~1 week.

7.6.3 Objective 3

- Using my collated list of references and research materials, look into the most optimal language to develop the tool in.
- Compare which open libraries to use in development.
- This will approximately take ~ 1 week.

7.6.4 Objective 4

- Design and develop a tool that collates user data from public online breach tools such as Have I been Pwned, and monitors users network to scan for active telemetry transmission from the user's system. Only for Windows users.
- This will use the information gathered from objective 1 to focus on the specified active telemetry processes and objective simply acts as a tool to visualise the background telemetry.
- This will take ~ 4 weeks.

7.6.5 Objective *5*

- Analyse the collected data, compare with existing methodologies from research that categorises data collection practises, and assess its security effectiveness in relation to email and password breaches.
- This will take ~ 4 weeks.

2. BACKGROUND RESEARCH AND PROJECT RATIONALE

By definition, stated officially by Cambridge University, telemetry is the 'process of collection information about objects that are far away and sending the information somewhere', (Cambridge Dictionary, 2024). On the surface level, stated by Microsoft, the reason this data acquisition is carried out is for product improvement and Microsoft do this by natively running their 'Required diagnostic data', (Microsoft.com, 2024).

However, in this present technological age where technology companies are more commercialised than ever, telemetry is under the guise of 'improving user experience' but its data collection is extremely excessive, and users often lack tangible evidence over what data is being acquired and sent to external servers.

The data sent back include device drivers, cookie history, location, email address, name and age, download history and much more, and one method developers use to collect this information is through the use of 'cookie banners' on websites. In most cases, when a user attempts to access a website, they are greeted with a 'cookie' agreement notice, these cookies are specifically known as consent notices, and recently websites now ask users for consent prior to cookie settings. Utz et al., 2019, discussed how "consent notices have become ubiquitous", yet most provide too little or too many options, and as a result this "fuels the habit to click any interaction element that causes the notice to go away instead of actively engaging with it and making an informed choice". This leads to a false interpretation of user consent, further stressing the fact that users are unintentionally leaking unconsented data.

Arastouie and Razzazi (2008) described these tools as 'spyware', in the sense that such software often installs themselves onto the user's device, which collect information that is sent back to the 'spy data base'. This includes online advertisements presented as banners, mask themselves as free software, and more often than not the intention of monitoring and collecting data is present within End User License Agreements (EULAs) immediately before installation. These EULAs are also present in mobile applications, and a group of researchers (Zimmeck et al., 2019) investigated the scale of privacy compliance to over a millions applications on Google Play Store, and found that many applications do not even have a privacy disclosure agreement, and the ones that do have a privacy policy are "often silent on the practices performed by apps" and found that "many developers are not accurately disclosing apps' privacy practises". This is not just telemetry; this is a complete violation of user privacy.

To summarise, this project bridges the gap between abstract privacy concerns and excessive data acquisition, and personalised evidence contributing to user awareness. The scope of this project will mainly focus on Windows telemetry through its active registries' responsible for acquisition of user

data, with an additional tool of linking 'Have I Been Pwned' (HIBP) API tool to identify any online breaches that may have compromised the user's email address.

3. METHODOLOGY AND TECHNICAL APPROACH

The overall methodology that will be used for this approach is agile development. Agile development is a form of an iterative development where the lifecycle of the project is composed of multiple sequential iterations, with each iteration being a "self-contained mini-project composed of activities such as requirement analysis, design, programming and test", as explained by Larman, 2004. In this section, these 'mini-projects' will be defined as **sprints.**

Each **sprint** will last two weeks, with experimental designs and testing being carried out throughout the lifecycle of this project. Ideally, the **sprints** will closely resemble the stated objectives in section 1.2, with further detail on the technical approach of the tool. The programming language that will be used is Python, as it has vast support for variety of libraries for API integration, most notably 'requests' library for retrieving and sending data and from an API which in this case will be Have I Been Pwned. To store the user's email securely, the python library 'hashlib' will be used to hash the email using a MD5 or SHA hash algorithm, ensuring confidentiality of data. Another library that will be used extensively in 'winreg', as it will expose the Windows registry API to the user, allowing for the tool to analyse and modify the registries. The application's graphical user interface (GUI) will be done using 'PyQt' to create a simple, yet modern and interactive interface because this tool is intended to be used for any user with or without technical knowledge of operating system registries, or telemetry.

Sprint 1 (Weeks 1-2): Foundation Research

This sprint will focus on intense research on relevant papers, journals, articles, academic literatures and existing open-source tools to build background knowledge of this project tool's rationale, and to establish a project scope and specifications. Google scholar will be used as the main reference point to gather the required research material.

Sprint 2 (Weeks 3-4): Implementing Registry Analyser

These weeks will focus on building the Windows registry analyser tool for active and running registries, the Python library 'winreg' will be used to allow communication between Windows registry API and the user. An open-source script that will be used as a benchmark for filtering the registries responsible for Windows telemetry is called 'HushWin', with a distributed copy found on GitHub.

Sprint 3 (Weeks 5-6): 'Have I Been Pwned' (HIBP) API Email Breach Integration

The objective of this sprint is to successfully integrate the HIBP API for email breach detection, where after the user inputs their email address it will successfully output any breaches, their email may have been involved in. The python library 'requests' will be used to establish the connection with the user and HIBP API. Another feature that will be added is incorporating a data structure to store the results securely, which will be implemented using 'hashlib'. This library will hash the users email input, and the tool will match the hashes for successful email retrieval.

Sprint 4 (Weeks 7-8): Telemetry Assessment Calculator

This feature focuses on developing a model to accumulate the results of the registry scans, and the email breach results and process them through a 'telemetry assessment' model that assign 'points' to each enabled setting in the registry and breached email, with a total out of 100 points, and collate the points to what the user will see as an 'Exposure Score'. The target of this is to help the user visualise their total online exposure, and the leakage of their personal data. This will be developed using native Python mathematical modules.

Sprint 5 (Weeks 9-10): GUI Integration

This sprint solely focuses on building a functioning application interface using Pythons library 'PyQt' as this library has extensive features compared to another popular GUI library called 'tkinter', such as custom widgets, multithreading support which would be useful for this project as multiple features will be running simultaneously, as well as offering an extensive layout management (Panicker, 2023). The main deliverable of this sprint is to initially offer a functioning prototype with a user interface.

Sprint 6 (Weeks 11 – 12): Data Collection and Analysis of Results

After acquiring data used by running this tool, the data will be compared with existing research from **Sprint 1** to compare results of the running registries and effectiveness of my tool compared to larger projects carried out in other research papers. The data will be acquired in a controlled virtual environment where the state of the virtual Windows 11 operating system will be fresh with no modifications on the system, and registries. The comparative data for data acquisition will involve comparing tool findings with manual registry inspection, and verifying HIBP analysis within my tool to known data breach databases, and finally collated on academic studies on Windows telemetry. Finally, this will be compiled into the final research paper.

4. EVALUATION PLAN

To quantitively and qualitatively measure the project's success, the aim and objectives will be closely used as a checklist to monitor progress, through frequent testing and comparing analysis with existing research. The following are evaluation objectives that will monitor the project's success.

Objective 1: Project Functionality

The aim of this objective is to simply measure if the project runs without fatal errors. This will include the registry analyser scanning all the predefined registries and returning and output, the HIBP integration returning a valid data breach known for compromising emails, and outputting a telemetry exposure score from 0-100 points.

Objective 2: Analysis Verification

This objective will verify the results of the projects analysis by comparing the results from the application tool to the results gathered from manual registry inspection carried out in a controlled environment. This will also be responsible for validating the outputted HIBP data breaches to existing breaches.

Objective 3: Verifying Results to Existing Research

This objective aims to verify the novelty of this project and measure the extent of its contribution to existing research, assessing if it accurately targeted the project scope. The results will be analysed with existing research such as Merzdovnik et al., 2017 who conducted a large-scale analysis of the effectiveness of tracker-blocking tools and Amoruso, 2024 who carried out a detailed analysis of Windows registries.

Objective 4: Usability of the tool

This objective simply aims to measure whether the final interface of the tool is functioning and usable, with no visual errors.

5. LSEPI CONSIDERATIONS

This tool will process personal data so must comply with the laws stated within the GDPR regarding data confidentiality, where "the controller and the processor shall implement appropriate technical and organisational measures to ensure a level of security appropriate to the risk", (GDPR, 2018). The tool must strictly be used for its only purpose of visualising telemetry from software and must not use that data for any other reason but the one stated, therefore only collect data necessary for this purpose. Users must provide informed consent for personal data to be used and processed explaining how it will be used and stored. Stored personal information must be anonymous and confidential (hashed or encrypted). If user would want to delete its data, there must be a clear option available for this request. This tool will only be used and ran locally, with the registry analysis confined to the local host machine, it will not scan systems the user does not own.

6. RISK ASSESSMENT

A major challenge that may be encountered during the development of this project will be in analysing Windows registries. This is due to the fact that the Windows operating system source code is not disclosed for public access, and despite the library featured earlier regarding an open API that links Windows registries to the user, there may still closed registries that are responsible for kernel-level operations that the tool will not have access to, potentially limiting the scope of my project in terms of analysing telemetry present in Windows registry files. The way this will be mitigated is by filtering out specific registries that can be accessed via 'winreg', and defining the project scope on those specific registries.

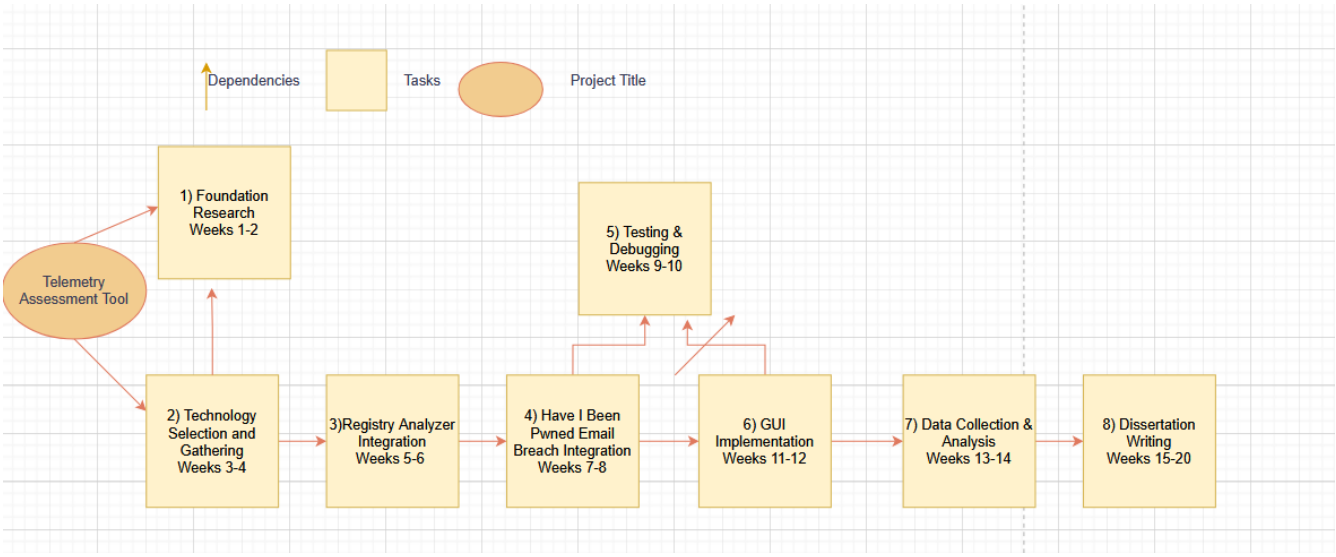
Due to the nature and ambiguity of the project theme, there are many features this project can explore, such as 'cookie banners', data sent through network traffic, and many more. However, due to deadline limitations the project scope mustn't be overly ambitions and as a result it should have a minimum of two features for necessary complexity. This will be mitigated by focusing the project scope only on the two defined features of Windows registry analyser and HIBP email breacher.

The users email input must be securely stored using hashing to ensure data confidentiality, so that it is not susceptible to data attacks, and so that the user can successfully retrieve the data whenever prompted. This will be mitigated by hashing the user emails only, keeping it minimal prevent hashing collisions, and encrypting the API calls.

Ethical and legal violation laws protect the user from mis-inputting sensitive information on dangerous websites and applications. This project will clearly state how the user's sensitive information will be used, stored, and processed and will be provided a clear option to delete their data at any point.

7. PROJECT PLAN AND TIMELINE

To visualise the project plan, this is a PERT chart with stated tasks, its time frame, and the link between the tasks.



The stated tasks in this graph follow the same structure as the methodology plan in section 3. It follows the same agile concept of using two-week sprints for each task, apart from the final 8th task of dissertation writing which is 4 weeks long.

The oval shape defines the project itself, the local telemetry assessment tool, the arrows show the links and dependencies to each task as many of the stated tasks and **sprints (**from section 3 in methodology and technical approach) progress depends on its preceding task. The square boxes highlight the task itself which are numbered in terms of order of completion, with the time frame written underneath.

Task 1 week 1 to week 2 begins on 13TH October 2025 till 26th October 2025. Task 2 weeks 3 to 4 is from October 27th till November 9th. Task 3 weeks 5 to 6 is from November 10th till November 24rd. Task 4 weeks 7 to 8 is from November 26th till December 11th. Task 5 weeks 9 to 10 is from December 29TH till January 5th 2026. Task 6 weeks 11 to 12 is from January 6th to January 20th. Task 7 weeks 13 to 14 is from January 21st to February 4th. Final task 8 weeks 15 to 20 is from March 5th.

REFERENCES

1. Microsoft.com. (2024). *Data collection summary for Windows*. [online] Available at: https://www.microsoft.com/en-us/privacy/data-collection-windows.
2. Cambridge Dictionary (2024). *telemetry*. [online] @CambridgeWords. Available at: https://dictionary.cambridge.org/dictionary/english/telemetry.
3. N. Arastouie and M. R. Razzazi, "Hunter: An Anti spyware for windows Operating System," *2008 3rd International Conference on Information and Communication Technologies: From Theory to Applications*, Damascus, Syria, 2008, pp. 1-5, doi: 10.1109/ICTTA.2008.4530281.
4. Zimmeck, S., Story, P., Smullen, D., Ravichander, A., Wang, Z., Reidenberg, J., Cameron Russell, N. and Sadeh, N. (2019). MAPS: Scaling Privacy Compliance Analysis to a Million Apps.
5. Utz, C., Degeling, M., Fahl, S., Schaub, F. and Holz, T. (2019). (Un)informed Consent: Studying GDPR Consent Notices in the Field ACM Reference Format. *(Un)informed Consent: Studying GDPR Consent Notices in the Field*. [online] doi:https://doi.org/10.1145/3319535.3354212.
6. Amoruso, Edward L., "Privacy and Security of the Windows Registry" (2024). Graduate Thesis and Dissertation 2023-2024. 118. https://stars.library.ucf.edu/etd2023/118
7. Larman, Craig (2004). Agile and Iterative Development: A Manager's Guide. Addison-Wesley. p. 10.
8. Panicker, A. (2023). *Tkinter vs PyQt: Choosing the Right GUI Library for Your Python Projects*. [online] DEV Community. Available at: https://dev.to/abpanic/tkinter-vs-pyqt-choosing-the-right-gui-library-for-your-python-projects-1oj0.
9. bitlogik (2021). *GitHub - bitlogik/HushWin: Shell script to disable all Windows capabilities about telemetry and tracking*. Available at: https://github.com/bitlogik/HushWin?tab=readme-ov-file
10. GDPR (2018). *Art. 32 GDPR – Security of processing | General Data Protection Regulation (GDPR)*. [online] General Data Protection Regulation (GDPR). Available at: https://gdpr-info.eu/art-32-gdpr/

APPENDIX B - Implementation Details

b.1 Implemented Windows Services and Justification

	Service Name	Function	Justification for Inclusion
1	DiagTrack (Connected User Experience and Telemetry Service)	This service is the primary telemetry transmission service, it collects and transmits diagnostic data, usage patterns, and performance metrics to Microsoft endpoints. If running, this service confirms active data transmission regardless of other configurations.	DiagTrack has been identified as the 'backbone' of Windows telemetry (Radar, 2022), and as such, disabling this service is the most common privacy hardening recommendation across third-party tools such as W10 Privacy.
2	dmwappushservice (Device Management Wireless Application Protocol Push Message Routing Service), also known as 'WAP'.	This service routes wireless application protocol messages received through device management channels, facilitating communication between devices and Microsoft. device management infrastructure. It is associated with device enrolment	Whilst this service is mostly responsible for enterprise device management, this service operates on consumer Windows installations and contributes to device identification telemetry.

		telemetry and enterprise data collection, allowing transmission of device compliance information.	
3	'DoSvc' (Delivery Optimization)	This is an active service that optimises network bandwidth for faster Windows updates through a peer-to-peer connection, and continuously stores system information as a result of it.	The activity of this service causes very large memory dumps and as a result contributes to continuous data transmission regarding network and operating system information.
4	'DPS' (Diagnostic Policy Service)	This service enables troubleshooting and resolution for Windows components, as well as coordinating diagnostic data collection policies. This service determines which diagnostic data is collected based on telemetry level settings acting as a policy enforcement mechanism.	Service status indicates whether diagnostic policies are actively enforced.
5	'WdiServiceHost' (Diagnostic Service Host)	This service hosts diagnostic services for components requiring problem	This service is a required dependency for DiagnosticExecutionService, so this service status indicates active diagnostic data system operations.

		detection and troubleshooting, providing a hosting infrastructure for multiple diagnostic data policies and systems.	
6	'WdiSystemHost' (Diagnostic System Host)	This service hosts system-level diagnostic services operating with elevated privileges and executes diagnostic functions requiring system-level access, allowing access to sensitive system information.	The escalated privilege activity means that this service activity indicates system wide diagnostic data collection beyond user-level information.
7	'PcaSvc' (Program Compatibility Assistant Service)	This service monitors program installations and sends compatibility issues to Microsoft endpoints. Logs are created as a result of this, including logs of application launch and crashes, program names and execution paths.	This is the telemetry service monitoring application usage; its status will indicate the active transmissions of detailed software usage patterns.
8	'WerSvc' (Windows Error Reporting Service)	This service collects and transmits error reports when programs crash or stop responding,	This is included due to historical controversy due to transmission of memory dumps, how these reports contain unintentional PII from application memory at the moment of the program crash.

		over-time creating crash dumps containing memory contents, stack traces and potentially sensitive data from application memory.	
9	'CDPUserSvc' (Connected Devices Platform User Service)	This service manages connected devices and sends its activity data, such as Bluetooth connections.	This service status indicates device usage activity and its history, if active will indicate transmission of device usage.
10	'OneSyncSvc' (Sync Host Service)	This service synchronises user data including their contacts, calendar and application settings with Microsoft cloud services. Transmits user generated content and synchronisation metadata to Microsoft endpoints.	Service activity indicates potential transmission of user created content and metadata beyond diagnostic telemetry.
11	'InventoryCollectorSvc' (Inventory Collector Service)	This service actively collects software and hardware inventory data (this is information about installed applications,	If enabled, will provide clear software and hardware telemetry evidence of transmitted information regarding the user's hardware specifications, and program information such as application history.

		hardware and software details) and is sent to Microsoft endpoints.	
12	'wuauserv' (Windows Update)	This service is responsible for downloading and installing system updates, transmitting diagnostic data to Microsoft endpoints.	Though a crucial service for running a safe Windows environment, this service has been known to cause high CPU usage and causing software issues such as crashes and bottlenecked performance. This provides clear evidence of how active telemetry transmission not only invades user privacy but inhibits optimal hardware performance.

b.2 Implemented Diagnostic Directories

C:\ProgramData\Microsoft\Diagnosis – this is the primary local storage location for Windows diagnostic data files including event trace logs generated by the DiagTrack service. The stored event log files may contain user paths, application names, and system configuration details.

C:\Windows\Temp\Diagnostics – this is the tool for troubleshooting problems and stores diagnostic data from previous troubleshoot runs. The data collected from these troubleshoots can vary from surface system information to personal user information.

%LocalAppData%\Diagnostics – similar to the first diagnostics folder but this is localised to the user account, storing significant diagnostic metadata for user-level applications which may unintentional PII within the diagnostic files.

C:\ProgramData\Microsoft\Windows\WER – this system-wide directory stores windows error reporting data that persists across user sessions from system services. As established in the services scanner component, these error dumps have a high risk of containing unintentional PII from application memory, demonstrating the full scope of error reporting data collection.

C:\ProgramData\Microsoft\Diagnosis\ETLLogs – this is the primary service used with DiagTrack as it gathers data about how the operating system and applications behave by storing event trace logs, which

is sent to Microsoft endpoints. These logs are gathered continuously (based on user privacy settings) and serve as high-value forensic artefacts of user and system information.

C:\ProgramData\Microsoft\Diagnosis\ETLLogs\AutoLogger – this is a system-level directory storing event tracing logs continuously, meaning the AutoLogger session runs immediately from system boot. This means the risk of storing PII is high, with information such as user interaction history and application usage being accumulated over time creating a large behavioural user profile.

C:\Windows\System32\LogFiles\WMI – this is the windows management instrumentation for log files, providing a comprehensive system management interface for recording trace logs. As it contains system configuration queries, this may also include user account information, installed software details and hardware inventory.

C:\Windows\System32\LogFiles\WMI\RtBackup - this is the folder used in conjunction with the WMI folder for storing real-time event trace sessions, unlike standard logs that record events to a permanent file this session stores user activity in real-time before committing official logs to the log folder. Collected information has a high risk of PII such as application usage, user information and more.

C:\Windows\Logs\CBS – this is a critical directory used for the component-based servicing engine responsible for managing the installation, uninstallation and updating of system components. This engine also records timestamps that can reveal user activity patterns and installed software states, the presence of these files indicates update history that has been transmitted to Microsoft endpoints.

%LocalAppData%\Microsoft\Windows\Explorer – this stores user specific cache for the windows file explorer for enhancing the desktop experience, though user-related information may be stored within the cache such as application usage patterns and browser history.

b.3 Monolithic and Microservices Software Architecture Definitions

This is a software design methodology that combines all of an application's components into a singular and inseparable unit, where unlike the layered architecture, the user interface, business logic and data access layers are all maintained as a singular unified unit. This methodology was rejected due to lack of independent testing, a crucial feature that was considered for this tool, due to the importance of forensic evidence preservation from each feature as they were all to be treated as individual evidence categories. Monolithic design would require full GUI instantiation for unit testing scanner logic, dramatically slowing test execution and complicating automated testing. Another considered alternative software design was the microservices architecture. This architecture is typically an application that is divided

into separate, independent services that communicate over a network. This would involve separate services for registry scanning, service enumeration, diagnostic analysis communicating through HTTP protocol. This architecture design was rejected as it is designed for distributed systems with independent scaling requirements, to which this project's tool has no distribution requirements and network communication would introduce unacceptable latency.